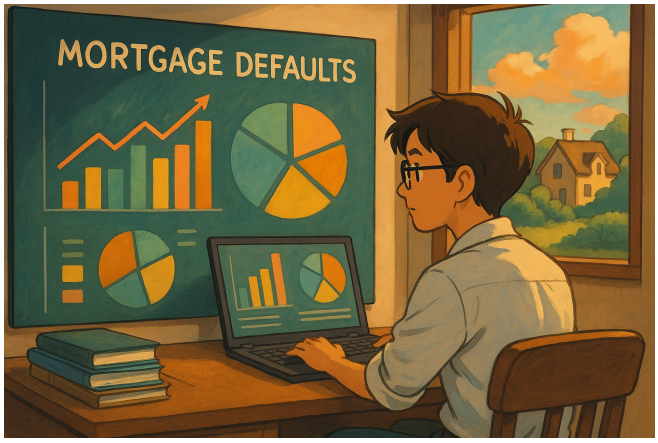
“The Risk Behind the Approval: Uncovering Default Before It Happens”

Leveraging machine learning to foresee mortgage defaults and inform early intervention strategies

---



EXECUTIVE SUMMARY:

This project aimed to develop a predictive model to assess mortgage default risk using Freddie Mac's historical loan-level dataset. Through comprehensive data analysis and feature engineering, key indicators of default were identified, including low credit scores (FICO), high debt-to-income ratios (DTI), high interest rates, and certain institutional and geographic factors. Categorical variables such as seller and servicer names were grouped based on historical default performance to improve model efficiency and interpretability. A logistic regression model was constructed as a transparent and interpretable baseline, offering a solid foundation for decision-making. In parallel, a more advanced Random Forest model was implemented to capture complex patterns and improve classification performance. While the Random Forest showed slightly higher recall and Fβ-score, the logistic regression model remained competitive and more straightforward to explain—making it valuable in regulatory and operational contexts. The final model was used to score active loans, identifying borrowers who show similar characteristics to historical defaulters. This enables early interventions such as refinancing offers or monitoring, helping reduce potential losses. In addition, business recommendations were developed based on model insights, including closer oversight of high-risk sellers and servicers, region-specific policy adjustments, and risk-based borrower segmentation.bOverall, the project demonstrates how predictive modeling can support more informed and proactive credit risk management, combining technical robustness with practical applicability in the mortgage lending industry.

Table of Contents

📌 Welcome to this notebook-style consultancy report prepared for Freddie Mac.
This notebook presents a consultancy-style report for Freddie Mac, using historical loan-level data to build a predictive model for mortgage defaults. The sections below walk through the full data science workflow supporting this goal.

Setup

This section prepares the coding environment by importing necessary tools for data processing, mapping, visualization, and machine learning. It also includes optional package installation commands and some basic display settings.

In [49]:
```
# Python package installer
#!pip install folium
#!pip install imblearn
#!pip install tensorflow
#!pip install tensorflow
```

In [50]:
```
# Add any additional libraries or submodules below
# ========== Core Libraries ==========
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import zipfile
# ========== Display and Mapping ==========
from IPython.display import display
import folium
from folium import Marker
from folium.features import DivIcon
import geopandas as gpd
# ========== Plotting Settings ==========
plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['figure.dpi'] = 80
# ========== Scikit-Learn: Preprocessing ==========
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
# ========== Scikit-Learn: Model and Pipeline ==========
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import DecisionBoundaryDisplay
# ========== Scikit-Learn: Model Selection ==========
from sklearn.model_selection import (
    train_test_split, GridSearchCV, RandomizedSearchCV,
    KFold, StratifiedKFold
)
# ========== Scikit-Learn: Metrics ==========
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    ConfusionMatrixDisplay, RocCurveDisplay, PrecisionRecallDisplay,
    roc_auc_score, make_scorer
)
# ========== Scikit-Learn: Feature Selection ==========
from sklearn.feature_selection import SelectFromModel
# ========== Imbalanced-Learn ==========
from imblearn.pipeline import Pipeline as ImPipeline
from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler
# ========== Custom Metrics ==========
from sklearn.metrics import fbeta_score
```

In [51]:
```
zip_file_path = "C:/Users/loren/OneDrive/Desktop/machine learning mortgage/freddiemac.zip"
csv_file_name = 'freddiemac.csv'

with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    with zip_ref.open(csv_file_name) as file:
        d = pd.read_csv(file)
```

```
C:\Users\loren\AppData\Local\Temp\ipykernel_45832\607634544.py:6: DtypeWarning: Columns (26,28) have mixed types. Specify dtype option on import or set low_memory=False.
  d = pd.read_csv(file)
```

1. Introduction

Background

In today's volatile economic environment, managing mortgage risk is more important than ever. For government-sponsored enterprises like **Freddie Mac**, whose mission is to promote stability and affordability in the housing market, understanding and mitigating default risk is critical. When a borrower fails to meet their loan obligations—commonly known as a **loan default**—the financial impact can ripple across stakeholders, from lenders and investors to insurers and policymakers. Accurately identifying which loans are likely to default empowers organizations to protect their portfolios, allocate resources more effectively, and reduce systemic risk. With billions of dollars tied up in mortgage-backed securities, even a small increase in default accuracy can translate into significant cost savings and strategic advantage.

This project leverages a rich dataset of **historical loan-level performance records** from Freddie Mac, containing granular information on borrower profiles, loan terms, property characteristics, and repayment outcomes. However, default prediction is not a straightforward task. Most loans in the dataset are fully paid or prepaid, while only a small proportion result in default—posing a **class imbalance challenge** that complicates analysis. Using advanced **machine learning classification techniques**, our goal is to build a model that can reliably flag high-risk loans early in the loan lifecycle. Beyond accuracy, we emphasize **interpretability**, ensuring that the insights generated can be trusted, understood, and acted upon by business users and decision-makers. The result is a data-driven solution designed not only to predict outcomes, but to **support proactive strategies and smarter lending decisions**.

Business Goal

Develop a **predictive classification model** that:

- Identifies loans at risk of **default**.
- Supports risk **mitigation** strategies and **early intervention**.
- Provides **interpretable** insights to support decision-making.

Consulting Objectives

🎯 The consulting objectives are:

1. **Predict Loan Default:** Build and validate a classification model to distinguish between prepaid and defaulted loans.
2. **Explain Key Drivers:** Identify the most influential features using interpretable machine learning techniques.
3. **Support Stakeholders:** Present actionable visual insights to guide Freddie Mac's internal risk review and policy design.

Data and Method Overview

Dataset Overview

📊 **Source:**
Freddie Mac Loan-Level Historical Performance Data

Sample:
The dataset has been cleaned and filtered to include only records of loans that are either **prepaid** or **defaulted**. It specifically focuses on loans originated between **2017 and 2020**, capturing their performance status—paid off, defaulted, or still active as of the cutoff date.

Target Variable:

- `loan_status = 0` : Prepaid
- `loan_status = 1` : Defaulted

The main task is to build a **classification model** to predict this binary loan status using available loan and borrower characteristics.

🔍 **Key Feature Types**

Feature Type	Variables	Description & Relevance
Borrower Info	`fico` , `cnt_borr` , `flag_fthb` , `dti` , `occpy_sts`	Captures borrower credit profile, status, and obligations. Lower FICO, higher DTI, or multiple borrowers may influence default likelihood.
Loan Characteristics	`cltv` , `ltv` , `orig_loan_term` , `int_rt` , `io_ind` , `program_ind` , `rr_ind` , `orig_upb`	Key indicators of loan structure. High LTV/CLTV, adjustable-rate structures, or large loan balances are associated with higher risk.
Property Info	`prop_type` , `zipcode` , `property_val` , `mi_pct` , `mi_cancel_ind`	Information about the mortgaged property. Property type or regional risk (e.g. via ZIP) may affect repayment behavior.
Origination Details	`channel` , `loan_purpose` , `ppmt_pnlty` , `prod_type`	Details at loan origination. Loans with prepayment penalties or certain purposes may have different performance characteristics.
Loan Identification	`id_loan` , `id_loan_rr` , `dt_first_pi` , `dt_matr` , `servicer_name` , `seller_name` , `flag_sc`	Operational identifiers and timestamps. Helpful for traceability and audit but not predictive.
Other	`st` , `cd_msa`	Geographic indicators such as state and metro area codes that may relate to regional economic risk.

📝 **Note:**
The majority of loans are classified as **prepaid**, resulting in an **imbalanced dataset**. This imbalance must be carefully addressed to avoid biased predictions.

Method Overview

This project follows a structured data science pipeline comprising four key phases, from initial data understanding through to final model interpretation. Each step is carefully designed to ensure the solution is both predictive and explainable for decision-makers. The methodology is divided into four key phases: **Data Understanding & Preprocessing, Baseline Model Development, Advanced Model Exploration, and Model Selection & Interpretation**, ensuring a systematic and reproducible approach.

1. **Data Understanding & Preprocessing**
   The process begins with a comprehensive **exploratory data analysis (EDA)** to understand the structure of the dataset, detect missing values, and analyze feature distributions. Necessary **preprocessing steps**, such as handling missing data, encoding categorical variables, and normalizing numerical features, are applied to ensure data consistency. Additionally, the dataset is **split into training and testing sets** to enable robust model evaluation.
2. **Baseline Model Development**
   A **Logistic Regression model** as baseline model is built for its simplicity, speed, and interpretability. Key performance metrics—Accuracy, Precision, Recall, AUC, and F1-Score—are used to evaluate the model, ensuring a balanced view, especially for imbalanced data. Initial feature selection is guided by domain knowledge and correlation analysis to identify important or redundant variables early on.
3. **Advanced Model Exploration**
   In this phase, we expanded beyond the baseline by developing and evaluating several advanced models to improve prediction accuracy. These included Support Vector Machines (SVM), Decision Trees, Random Forests, Neural Networks, and Ensemble methods. Each model was fine-tuned and assessed using key performance metrics, allowing for a comprehensive comparison against the logistic regression baseline. This exploration helped identify the most effective predictive strategy for distinguishing between loan defaults and prepayments.
4. **Model Selection & Interpretation**
   In the final stage, the best-performing model is chosen by balancing accuracy, simplicity, and interpretability. Key features are analyzed to understand their impact on predictions, making the model more transparent and useful for real-world decision-making.

By following this structured methodology, the project ensures that the developed model is robust, interpretable, and capable of providing actionable insights into loan default prediction. This process is also illustrated through the following workflow:

2. Data Exploration and Preprocessing

Data Understanding

This section provides an overview of the dataset by first displaying its structure and then generating summary statistics. It confirms that the dataset contains 200,000 entries and 33 columns, with various data types including integers, floats, and categorical objects.

In [52]:
```
# Identify the structure of the dataset
display(d.info())
 # Summary statistics
display(d.describe().round(2))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 33 columns):
 #   Column          Non-Null Count   Dtype  
---  ------          --------------   -----  
 0   fico            200000 non-null  int64  
 1   dt_first_pi     200000 non-null  int64  
 2   flag_fthb       200000 non-null  object 
 3   dt_matr         200000 non-null  int64  
 4   cd_msa          181072 non-null  float64
 5   mi_pct          200000 non-null  int64  
 6   cnt_units       200000 non-null  int64  
 7   occpy_sts       200000 non-null  object 
 8   cltv            200000 non-null  int64  
 9   dti             200000 non-null  int64  
 10  orig_upb        200000 non-null  int64  
 11  ltv             200000 non-null  int64  
 12  int_rt          200000 non-null  float64
 13  channel         200000 non-null  object 
 14  ppmt_pnlty      200000 non-null  object 
 15  prod_type       200000 non-null  object 
 16  st              200000 non-null  object 
 17  prop_type       200000 non-null  object 
 18  zipcode         200000 non-null  int64  
 19  id_loan         200000 non-null  object 
 20  loan_purpose    200000 non-null  object 
 21  orig_loan_term  200000 non-null  int64  
 22  cnt_borr        200000 non-null  int64  
 23  seller_name     200000 non-null  object 
 24  servicer_name   200000 non-null  object 
 25  flag_sc         7531 non-null    object 
 26  id_loan_rr      2402 non-null    object 
 27  program_ind     200000 non-null  object 
 28  rr_ind          2402 non-null    object 
 29  property_val    200000 non-null  int64  
 30  io_ind          200000 non-null  object 
 31  mi_cancel_ind   200000 non-null  object 
 32  loan_status     200000 non-null  object 
dtypes: float64(2), int64(13), object(18)
memory usage: 50.4+ MB
None
```

	fico	dt_first_pi	dt_matr	cd_msa	mi_pct	cnt_units	cltv	dti	orig_upb	ltv	int_rt	zipcode	orig_loan_term	cnt_borr	property_val
count	200000.00	200000.00	200000.00	181072.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00	200000.00
mean	751.63	201872.54	204589.51	30225.97	7.19	1.03	74.67	46.48	250800.00	74.37	4.09	55412.85	327.00	1.48	1.89
std	139.87	117.33	570.81	11246.32	11.94	0.23	17.87	105.67	130510.35	17.40	0.76	29501.10	67.59	0.51	0.42
min	458.00	201702.00	202502.00	10180.00	0.00	1.00	5.00	1.00	11000.00	5.00	0.00	600.00	96.00	1.00	1.00
25%	719.00	201803.00	204705.00	19430.00	0.00	1.00	66.00	28.00	151000.00	66.00	3.62	30300.00	360.00	1.00	2.00
50%	758.00	201903.00	204807.00	31540.00	0.00	1.00	80.00	36.00	227000.00	80.00	4.12	55100.00	360.00	1.00	2.00
75%	787.00	202003.00	204909.00	39580.00	12.00	1.00	88.00	43.00	330000.00	86.00	4.62	84000.00	360.00	2.00	2.00
max	9999.00	202205.00	206303.00	49740.00	999.00	4.00	999.00	999.00	1088000.00	999.00	6.75	99900.00	516.00	5.00	9.00

👉 **According to the information above:**

- The column types include:
  - **13 numeric columns** of type `int64`, such as:
    `fico`, `dt_first_pi`, `dt_matr`, `mi_pct`, `cnt_units`, `cltv`, `dti`, `orig_upb`, `ltv`, `zipcode`, `orig_loan_term`, `cnt_borr`, `property_val`
  - **2 numeric columns** of type `float64`:
    `cd_msa`, `int_rt`
  - **18 categorical columns** of type `object`, such as:
    `flag_fthb`, `occpy_sts`, `channel`, `ppmt_pnlty`, `prod_type`, `st`, `prop_type`, `loan_purpose`, `seller_name`, `servicer_name`, `flag_sc`, `program_ind`, `rr_ind`, `io_ind`, `mi_cancel_ind`, `id_loan_rr`, `loan_status`, `id_loan`
- The **target variable** is `loan_status`, which classifies loans as either *defaulted*, *prepaid*, or *active*.
- While all columns show 200,000 non-null entries in most fields, **some contain placeholder values like `999`, `9999`, or `9`**, which indicate *Not Available, Not Applicable or unknown values*. These will need to be handled further in the cleaning phase.
- Initial summary statistics indicate:
  - `fico` (credit score) ranges from **458 to 9999**, where **9999 indicates invalid entries**.
  - Variables like `cltv` and `dti` also have **extreme or placeholder values** (e.g. `999`).

➡️ These findings suggest that while the dataset has no technically missing values (`NaN`), it does contain **invalid placeholder values** that must be treated during data preprocessing.

Data Cleaning

Data cleansing, or data cleaning, is the process of organizing and refining raw data to make it suitable for analysis. It involves correcting errors, handling missing or incomplete entries, eliminating irrelevant data, and transforming variables when necessary. This step is crucial in machine learning, as it enhances model performance by minimizing noise and inconsistencies. If not done properly, unclean data can lead to overly complex and less interpretable models that are more prone to overfitting. Even small inaccuracies or noise in the data can significantly reduce a model's accuracy and effectiveness.

The cleaning process include:

- Duplicate value treatment
- Noise treatment
- Missing value treatment
- Outlier treatment
- Feature transformartion

All the cleaning proces follow this cleaning flow:

In [53]:
```
#Check for fully duplicated rows
duplicate_rows = d.duplicated()
print(f"Number of fully duplicated rows: {duplicate_rows.sum()}") #NO FULLY DUPLICATED ROWS
```

```
#Check for duplicated Loan IDs (should be unique)
if 'id_loan' in d.columns:
    duplicate_ids = d['id_loan'].duplicated()
    print(f"Number of duplicated 'id_loan' entries: {duplicate_ids.sum()}") #NO DUPLICATED LOANS
else:
    print("Column 'id_loan' not found in the dataset.")
```

```
Number of fully duplicated rows: 0
Number of duplicated 'id_loan' entries: 0
```

Verified that there are no fully duplicated rows and that all `id_loan` entries are unique.

In [54]:
```
#count how many 9999 in fico
count_999_fico = (d['fico'] == 9999).sum()
print("Number of 9999 in 'fico':", count_999_fico)
#replace them with NAs
d['fico'] = d['fico'].replace(9999, np.nan)
```

```
Number of 9999 in 'fico': 41
```

Replaced invalid FICO values (`9999`) with `NaN` (41 in total). These will be handled after the dataset split: removed in the binary subset, retained and imputed in the active one (in order to perform prediction on all the active loans)

In [55]:
```
#convert 'dt_first_pi' and 'dt_matr' from YYYYMM string format to datetime
d['dt_first_pi'] = pd.to_datetime(d['dt_first_pi'].astype(str), format='%Y%m')
d['dt_matr'] = pd.to_datetime(d['dt_matr'].astype(str), format='%Y%m')
#minimum start date
min_date = d['dt_first_pi'].min()
#create a new column to track when mortgage started
d['start_months'] = ((d['dt_first_pi'].dt.year - min_date.year) * 12 +
                     (d['dt_first_pi'].dt.month - min_date.month)).astype(int)
```

Converted `dt_first_pi` and `dt_matr` to datetime format. Created a new numeric variable `start_months` representing the number of months since the earliest mortgage start date. After this knowing `start_months` and `orig_loan_term` we can drop `dt_first_pi` and `dt_matr` .

In [56]:
```
#count the number of entries with value '9' (not available) in 'flag_fthb'
count_9_flag_fthb = (d['flag_fthb'] == '9').sum()
print(f"Number of '9' entries in 'flag_fthb': {count_9_flag_fthb}")
#there are no unavaible data in flag_fthb
```

```
Number of '9' entries in 'flag_fthb': 0
```

In [57]:
```
#count the number of entries with value 999 in 'mi_pct'
count_999_mi_pct = (d['mi_pct'] == 999).sum()
print(f"Number of 999 entries in 'mi_pct': {count_999_mi_pct}")
print(f"Loan status of the 999 value: {d.loc[d['mi_pct'] == 999, 'loan_status']}")
d = d[d['mi_pct'] != 999]
```

```
Number of 999 entries in 'mi_pct': 1
Loan status of the 999 value: 167716    prepaid
Name: loan_status, dtype: object
```

Removed one entry with invalid `mi_pct = 999` , as it was the only case and not impactful on the analysis (prepaid loan).

In [58]:
```
#count 99 in cnt_units
count_99_cnt_units = (d['cnt_units'] == 99).sum()
print(f"Number of 99 entries in 'cnt_units': {count_99_cnt_units}")
# Transform into categorical data type
units_map = {
    1: 'One Unit',
    2: 'Two Units',
    3: 'Three Units',
    4: 'Four Units',
    99: 'Not Available'}
d['cnt_units'] = d['cnt_units'].map(units_map)
```

```
Number of 99 entries in 'cnt_units': 0
```

Mapped `cnt_units` to categorical labels and labeled `99` as "Not Available".

In [59]:
```
#9 in occpy_sts
count_9_occpy_sts = (d['occpy_sts'] == '9').sum()
print(f"Number of '9' entries in 'occpy_sts': {count_9_occpy_sts}")#0 -> good
```

```
Number of '9' entries in 'occpy_sts': 0
```

In [60]:
```
#count  999 cltv
count_999_cltv = (d['cltv'] == 999).sum()
print(f"Number of 999 entries in 'cltv': {count_999_cltv}")
#6 Na "999", 1 is prepaid and 5 are active, we delete the prepaid and the later imputed the active
d = d[~((d['cltv'] == 999) & (d['loan_status'] == 'prepaid'))]
#replace 999 with NaN in 'cltv'
d['cltv'] = d['cltv'].replace(999, np.nan)
```

```
Number of 999 entries in 'cltv': 6
```

Removed prepaid loans with invalid `cltv = 999` and replaced remaining `999` values (from active loans) with `NaN` for later imputation.

In [61]:
```
#count 999 entries in 'dti'
count_999_dti = (d['dti'] == 999).sum()
print(f"Number of 999 entries in 'dti': {count_999_dti}")
d['dti'] = d['dti'].replace(999,70)
```

```
Number of 999 entries in 'dti': 2412
```

Replaced `dti = 999` (indicating values >65%) with `70` as a placeholder to retain information without using `NaN` .

In [62]:
```
#channel:
count_9_channel = (d['channel'] == '9').sum()
print(f"Number of '9' entries in 'channel': {count_9_channel}")#all good
```

```
Number of '9' entries in 'channel': 0
```

In [63]:
```
# Count the number of unique states in 'st'
num_states = d['st'].nunique()
print(f"Number of unique states in 'st': {num_states}")
# Cut the zip code into 3 digits only
d['zipcode'] = d['zipcode'].astype(str).str[:3]
# Count the number of unique ZIP codes
num_zipcodes = d['zipcode'].nunique()
print(f"Number of unique ZIP codes: {num_zipcodes}")
```

```
Number of unique states in 'st': 54
Number of unique ZIP codes: 804
```

Truncated ZIP codes to 3 digits for regional aggregation and confirmed the number of unique states and ZIP codes.

In [64]:
```
# Count the number of 99 entries in 'prop_type'
count_99_prop_type = (d['prop_type'] == '99').sum()
print(f"Number of 99 entries in 'prop_type': {count_99_prop_type}")# good
```

```
Number of 99 entries in 'prop_type': 0
```

In [65]:
```
# Count the number of '9' entries in 'loan_purpose'
count_9_loan_purpose = (d['loan_purpose'] == '9').sum()
print(f"Number of '9' entries in 'loan_purpose': {count_9_loan_purpose}")#good
```

```
Number of '9' entries in 'loan_purpose': 0
```

In [66]:
```
#seller name
#unique seller names
num_sellers = d['seller_name'].nunique()
print(f"Number of unique seller names: {num_sellers}")
#check redundancy between seller name and servicer name
#in csv file other for servicer and seller have differen name (we need to treat them as the same)

# Create temporary copies with unified 'Other' label
seller_clean = d['seller_name'].replace('Other sellers', 'Other')
servicer_clean = d['servicer_name'].replace('Other servicers', 'Other')

# Count how many rows have different values (treating 'Other' as the same)
count_different = (seller_clean != servicer_clean).sum()
print(f"Number of rows where seller_name and servicer_name differ (treating 'Other' as equal): {count_different}")
# Count cases where seller is 'Other' but servicer is not
count_other_seller_only = ((seller_clean == 'Other') & (servicer_clean != 'Other')).sum()
print(f"Rows where seller is 'Other' but servicer is not: {count_other_seller_only}")
#so in around 45% of the cases the servicer and seller are different
#servicer usually is more precise (we know name of seller but not servicer)
#I thought that if they were usually similar we could have dropped one. Does not seem to be the case.
```

```
Number of unique seller names: 49
Number of rows where seller_name and servicer_name differ (treating 'Other' as equal): 89233
Rows where seller is 'Other' but servicer is not: 23781
```

Standardized "Other" labels and found that seller and servicer names differ in ~45% of the dataset. Since they provide distinct information, both features were retained.

In [67]:
```
#'flag_sc' values to only 'Y' or 'N' (we already have Y but N are Nan)
d['flag_sc'] = d['flag_sc'].apply(lambda x: 'Y' if x == 'Y' else 'N')
```

In [68]:
```
#convert 'id_loan_rr' to 'Y' if populated, 'N' if NaN
d['id_loan_rr'] = d['id_loan_rr'].fillna('N')
#convert any value that is not 'N' to 'Y'
d['id_loan_rr'] = d['id_loan_rr'].apply(lambda x: 'N' if x == 'N' else 'Y')
```

In [69]:
```
#'9' entries in 'program_ind'
count_9_program = (d['program_ind'] == '9').sum()
print(f"Number of '9' entries in 'program_ind': {count_9_program}")#huge number -> DROP
```

```
Number of '9' entries in 'program_ind': 184179
```

In [70]:
```
#HARP loan: rr_ind == 'Y' AND ltv > 80
d['harp_loan'] = d.apply(lambda row: 'Y' if row['rr_ind'] == 'Y' and row['ltv'] > 80 else 'N', axis=1)
#count how many HARP loans
num_harp_loans = (d['harp_loan'] == 'Y').sum()
print(f"Number of HARP loans: {num_harp_loans}")
```

```
Number of HARP loans: 544
```

Created a new `harp_loan` column based on `rr_ind` and `ltv > 80`. Dropped `rr_ind` as it was redundant with `id_loan_rr`. Only a small number (544) of HARP loans were identified.

In [71]:
```
#number of '9' entries in 'property_val'
count_9_property_val = (d['property_val'] == 9).sum()
print(f"Number of '9' entries in 'property_val': {count_9_property_val}")
# transform into categorical data
val_map = {
    "1": "ACE Loans",
    "2": "Full Appraisal",
    "3": "Other Appraisals",
    "4": "ACE + PDR",
    "9": "Not Available"
}
d['property_val'] = d['property_val'].astype(str).map(val_map)
```

```
Number of '9' entries in 'property_val': 125
```

Mapped `property_val` codes to categorical labels, keeping `9` as "Not Available".

In [72]:
```
# Show unique values in 'io_ind'
unique_io_ind = d['io_ind'].unique()
print(f"Unique values in 'io_ind': {unique_io_ind}") #all N ->DROP
```

```
Unique values in 'io_ind': ['N']
```

In [73]:
```
# Count the number of '9' entries in 'mi_cancel_ind'
count_9_mi_cancel = (d['mi_cancel_ind'] == '9').sum()
print(f"Number of '9' entries in 'mi_cancel_ind': {count_9_mi_cancel}") #0 -> GOOD
#categorical data type to handle easily
mi_cancel_map = {
    'Y': 'Canceled',
    'N': 'Not Canceled',
    '7': 'No Insurance',
    '9': 'Not Disclosed'}
d['mi_cancel_ind'] = d['mi_cancel_ind'].map(mi_cancel_map)
```

```
Number of '9' entries in 'mi_cancel_ind': 0
```

Mapped `mi_cancel_ind` to categorical labels; no '9' values were present in the data.

In [74]:
```
# Transform cnt_borr into categorical data (object)
d['cnt_borr'] = d['cnt_borr'].apply(lambda x: 'One Borrower' if x == 1 else 'Multiple Borrowers')
```

In [75]:
```
#drop end date (redundant)
d = d.drop(columns=['dt_matr'])
#drop start date (we have new col start months)
d = d.drop(columns=['dt_first_pi'])
#drop the 'cd_msa' column (10% of missing data, zip and state as spatial data)
d = d.drop(columns=['cd_msa'])
#drop the 'ltv' column (reduntant with cltv)
d = d.drop(columns=['ltv'])
#drop the 'ppmt_pnlty' column (only N)
d = d.drop(columns=['ppmt_pnlty'])
#drop the 'prod_type' column (all the obs are fixed rate mortages)
d = d.drop(columns=['prod_type'])
#drop the 'id_loan' column (unique for each obs)
d = d.drop(columns=['id_loan'])
#drop the 'program_ind' column (many not-available data)
d = d.drop(columns=['program_ind'])
#drop the 'rr_ind' column (redundant with id_loan_rr)
d = d.drop(columns=['rr_ind'])
#drop the 'io_ind' column (all fully amortizing from the start)
d = d.drop(columns=['io_ind'])
```

Dropped redundant or uninformative columns, including dates, unique IDs, constant-value fields, and features with excessive missing data or better alternatives available. In our dataset we have only mortgages that are fixed rate and amortizing from the start.

In [76]:
```
# Count the frequency of each loan status
loan_status_counts = d['loan_status'].value_counts(dropna=False)
print("Loan status distribution:")
print(loan_status_counts)
```

```
Loan status distribution:
loan_status
prepaid    125957
active      73295
default       746
Name: count, dtype: int64
```

The dataset contains around 200,000 loans, of which 125,958 were prepaid, 73,295 are still active, and only 746 defaulted. Given the project's focus, we restrict our (initial) analysis to prepaid and defaulted loans, resulting in a highly imbalanced binary classification task.

In [77]:
```
# Keep only prepaid and default loans
d_binary = d[d['loan_status'].isin(['prepaid', 'default'])].copy()
# Create binary target: 1 = default, 0 = prepaid
d_binary['loan_status'] = d_binary['loan_status'].apply(lambda x: 1 if x == 'default' else 0)
# Filter out invalid FICO scores
d_binary = d_binary.dropna(subset=['fico']) #as said before drop NAs from FICO from binary
```

In [78]:
```
#dataset for active loans
d_active = d[d['loan_status'] == 'active'].copy()
# Because there is missing value on the 'fico' and 'cltv', we trying to make imputation on those variable
# Impute missing values in 'fico' and 'cltv' using median
d_active['fico'] = d_active['fico'].fillna(d_active['fico'].median())
d_active['cltv'] = d_active['cltv'].fillna(d_active['cltv'].median())
print(d_active.shape[0]) #number of observation in the active dataset
```

```
73295
```

Splitting Train and Test Data

This section focuses on splitting the dataset into training and testing subsets for model development and evaluation. First, the feature variables (`X`) are separated from the target variable (`y`), which is the loan status. A stratified train-test split is applied using a test size of 20%, ensuring that the distribution of the target variable (i.e., defaulted vs. prepaid loans) remains consistent across both subsets. A fixed random state (42) is set for reproducibility of the results.

After splitting, the training features and labels (`X_train` and `y_train`) are concatenated into a single DataFrame named `eda_train` . This combined dataset is intended for exploratory data analysis (EDA), which helps in understanding patterns, trends, and relationships in the training data before proceeding to feature engineering or model training.

In [79]:
```
#features (X), target (y)
X = d_binary.drop(columns=['loan_status'])
y = d_binary['loan_status']
# stratified split on loan status
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

In [80]:
```
#dataset for EDA
eda_train = pd.concat([X_train, y_train], axis=1)
eda_train.head()
df = pd.DataFrame(eda_train)
```

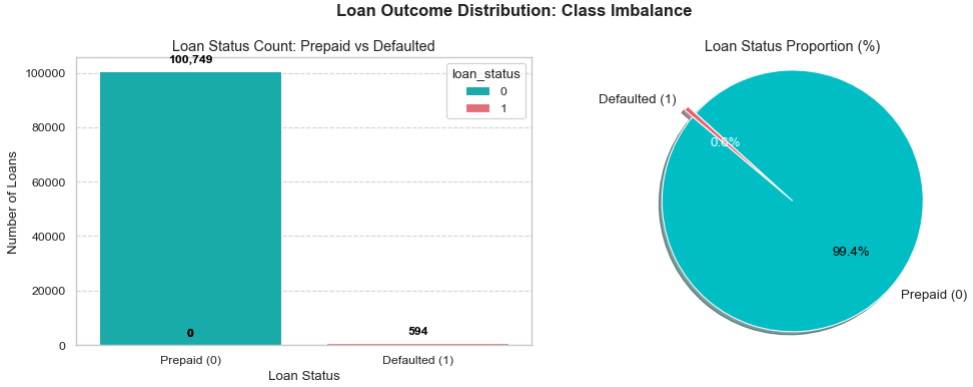
Exploratory Data Analysis (EDA)

To gain a better understanding of the data before modeling, we begin with Exploratory Data Analysis (EDA). This step helps uncover important patterns, detect anomalies, explore relationships between features, and evaluate the distribution of the target variable. It sets the foundation for making informed decisions in the feature engineering and modeling phases.

1. 🔍 Loan Outcome Distribution: Class Imbalance Overview

Purpose: To visualize class imbalance in the target variable `loan_status` .

In [81]:
```
# Create 2-subplot layout with vibrant colors and annotations
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
# Calculate percentage
loan_counts = eda_train['loan_status'].value_counts()
loan_percent = eda_train['loan_status'].value_counts(normalize=True) * 100
labels = ['Prepaid (0)', 'Defaulted (1)']
# Brighter color palette
colors = ['#00c2c7', '#f95d6a']
# Barplot with annotation
bars = sns.barplot(x=loan_counts.index, y=loan_counts.values,
                   hue=loan_counts.index, palette=colors, ax=axes[0])
axes[0].set_title('Loan Status Count: Prepaid vs Defaulted', fontsize=13)
axes[0].set_xlabel('Loan Status')
axes[0].set_ylabel('Number of Loans')
axes[0].set_xticks(range(len(labels)))         # Set fixed tick positions
axes[0].set_xticklabels(labels)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)
# Add annotation text on bars
for bar in bars.patches:
    height = bar.get_height()
    axes[0].annotate(f'{height:,.0f}',
                     xy=(bar.get_x() + bar.get_width() / 2, height),
                     xytext=(0, 8), textcoords='offset points',
                     ha='center', fontsize=11, fontweight='bold', color='black')
# Pie chart with explode and annotation
explode = (0, 0.08)
wedges, texts, autotexts = axes[1].pie(
    loan_percent, labels=labels, colors=colors, autopct='%1.1f%%',
    startangle=140, explode=explode, shadow=True, textprops={'fontsize': 12})

for i, a in enumerate(autotexts):
    a.set_color('white' if i == 1 else 'black')  # highlight default
axes[1].axis('equal')
axes[1].set_title('Loan Status Proportion (%)', fontsize=13)
# Final title
plt.suptitle('Loan Outcome Distribution: Class Imbalance', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```



🧠 Insights:

Extreme Class Imbalance:

- From the bar chart, we see that almost all loans are prepaid (label 0).
- Only a tiny fraction is defaulted (label 1) — visually almost invisible in count.
- Only 0.6% are Defaulted:

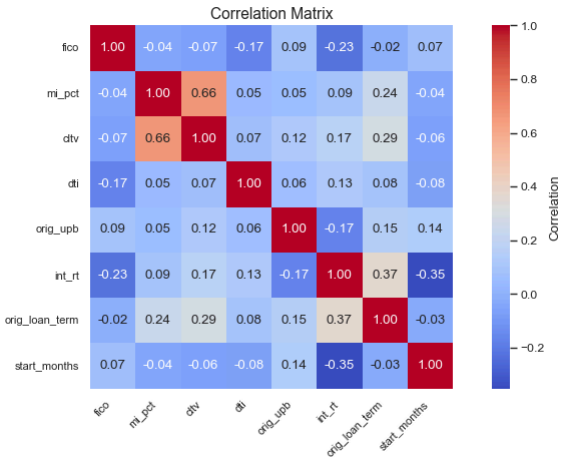
The pie chart emphasizes the imbalance further, showing 99.4% prepaid vs 0.6% defaulted.

Implications for Modeling:

- This class imbalance must be carefully addressed during modeling.
- Techniques like resampling, class weighting, or anomaly detection frameworks might be necessary.
- Accuracy alone will be misleading — models may predict "prepaid" for all loans and still look good.

2. 📊 Correlation Matrix

In [82]:
```
numeric_df = eda_train.select_dtypes(include='number')
numeric_df = numeric_df.drop(columns=['loan_status'])
cor_matrix = numeric_df.corr()
plt.figure(figsize=(10, 6))
sns.heatmap(
    cor_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    square=True,
    cbar_kws={'label': 'Correlation'})
plt.xticks(fontsize=10, rotation=45, ha='right')
plt.yticks(fontsize=10)
plt.title("Correlation Matrix", fontsize=14)
plt.tight_layout()
plt.show()
```



🧠 **Insights:**

The heatmap above shows the correlation coefficients between the numerical variables in the training set.

- **Strong positive correlation** is observed between `mi_pct` (mortgage insurance percentage) and `cltv` (combined loan-to-value ratio), with a coefficient of **0.66**. This is expected, as higher CLTV values typically require greater mortgage insurance coverage.
- **Moderate correlations**:
  - `int_rt` (interest rate) is positively correlated with `orig_loan_term` (0.37) and `cltv` (0.17), and negatively correlated with `start_months` (−0.35), possibly reflecting temporal trends in interest rates.
  - `orig_loan_term` shows a modest correlation with `cltv` (0.29), suggesting that longer loan terms are associated with higher CLTVs.
- **Weak correlations** (magnitude < 0.2) are present between most other variables. For instance, `fico` (credit score) shows only weak relationships with other features.
- **Negative correlations**:
  - `fico` is negatively correlated with `int_rt` (−0.23) and `dti` (−0.17), indicating that higher credit scores are generally associated with lower interest rates and debt-to-income ratios.
  - `start_months` is negatively correlated with `int_rt` (−0.35), suggesting a potential decrease in interest rates over time.

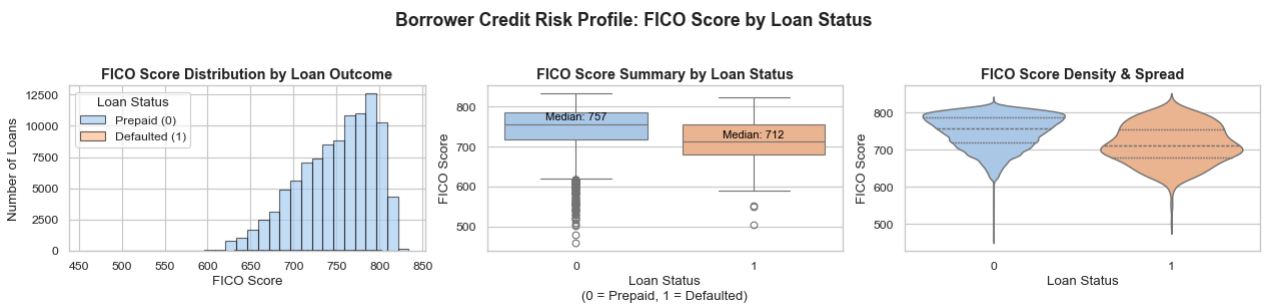
Most relationships are weak to moderate, indicating limited multicollinearity. However, stronger correlations, such as between `mi_pct` and `ltv`, should be considered when selecting features for the final model.

2. 💳 Borrower Credit Risk Profile: FICO Score by Loan Status

Purpose: To analyze how borrower credit scores (`fico`) vary across loan outcomes.

In [83]:
```
# Ensure 'fico' is numeric and drop rows where conversion fails
eda_train['fico'] = pd.to_numeric(eda_train['fico'], errors='coerce')

# Split FICO scores by loan status
fico_prepaid = eda_train[eda_train['loan_status'] == 0]['fico']
fico_defaulted = eda_train[eda_train['loan_status'] == 1]['fico']

# Create figure with 3 subplots
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# 1. Histogram
axes[0].hist(fico_prepaid, bins=30, alpha=0.6, label='Prepaid (0)', color='#a1c9f4', edgecolor='black')
axes[0].hist(fico_defaulted, bins=30, alpha=0.6, label='Defaulted (1)', color='#ffb482', edgecolor='black')
axes[0].set_title('FICO Score Distribution by Loan Outcome', fontsize=13, fontweight='bold')
axes[0].set_xlabel('FICO Score')
axes[0].set_ylabel('Number of Loans')
axes[0].legend(title='Loan Status')

# 2. Boxplot with median annotations
# Boxplot for loan_status = 0 and 1 separately
# First, convert loan_status to string for better legend handling
eda_train['loan_status_str'] = eda_train['loan_status'].astype(str)

# Now use hue
sns.boxplot(
    data=eda_train,
    x='loan_status_str',
    y='fico',
    hue='loan_status_str',
    palette=['#a1c9f4', '#ffb482'],
    ax=axes[1],
    legend=False
)
medians = eda_train.groupby('loan_status')['fico'].median()
for i, median in enumerate(medians):
    axes[1].text(i, median + 10, f'Median: {int(median)}', ha='center', fontsize=10, color='black')
axes[1].set_title('FICO Score Summary by Loan Status', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Loan Status\n(0 = Prepaid, 1 = Defaulted)')
```

```
axes[1].set_ylabel('FICO Score')

# 3. Violin Plot
sns.violinplot(
    data=eda_train,
    x='loan_status_str',
    y='fico',
    hue='loan_status_str',
    palette=['#a1c9f4', '#ffb482'],
    ax=axes[2],
    inner='quart',
    legend=False
)
axes[2].set_title('FICO Score Density & Spread', fontsize=13, fontweight='bold')
axes[2].set_xlabel('Loan Status')
axes[2].set_ylabel('FICO Score')

# Title and layout adjustment
plt.suptitle('Borrower Credit Risk Profile: FICO Score by Loan Status', fontsize=16, fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



💡 **Insights:**

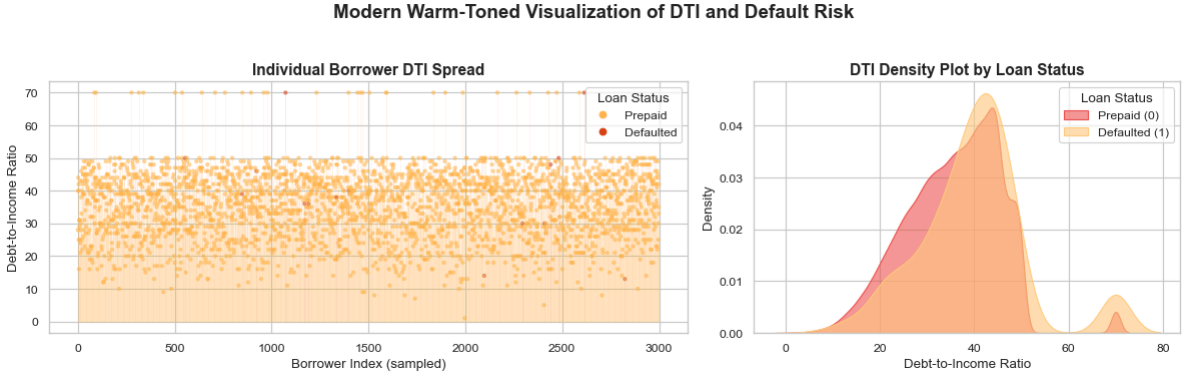
The FICO score emerges as a **highly discriminative feature** in understanding borrower credit behavior, with clear separation in distributions between those who prepaid and those who defaulted on their loans. Borrowers with **higher FICO scores (median ~757)** are substantially more likely to repay early, while those with **lower FICO scores (median ~711)** show significantly increased default risk. This pattern is consistent across histogram, boxplot, and violin visualizations, which collectively reveal both central tendency and variability in borrower credit profiles.

These findings align with financial literature affirming credit scores as strong predictors of loan performance. For instance, **Avery et al. (2000)** found that FICO scores are highly correlated with delinquency and default outcomes, and remain predictive even after accounting for other borrower characteristics. Thus, FICO score should be prioritized in risk models and potentially used as a threshold or binned feature for interpretability in credit scoring applications.

3. 📈 Debt-to-Income Ratio and Default Risk

Purpose: Understand the relationship between `dti` and loan default.

In [84]:
```
# Reuse color definitions
left_color = '#ffb74d'             # orange for stripplot
strip_color_defaulted = '#d84315'  # darker for defaulted
right_color_prepaid = '#ef5350'    # red for prepaid density
right_color_defaulted = '#ffcc80'  # light orange for defaulted density

# Create figure with 1 strip plot + 1 density plot
fig, axes = plt.subplots(1, 2, figsize=(15, 5), gridspec_kw={'width_ratios': [1.5, 1]})

# 1. Strip (lollipop-style) plot
sample_df = eda_train.iloc[:3000].copy()
colors = sample_df['loan_status'].map({0: left_color, 1: strip_color_defaulted})

axes[0].vlines(x=range(len(sample_df)), ymin=0, ymax=sample_df['dti'],
               color=colors, alpha=0.15, linewidth=0.5)
axes[0].scatter(x=range(len(sample_df)), y=sample_df['dti'], c=colors, s=8, alpha=0.5)
axes[0].set_title("Individual Borrower DTI Spread", fontsize=14, fontweight='bold')
axes[0].set_ylabel("Debt-to-Income Ratio")
axes[0].set_xlabel("Borrower Index (sampled)")
axes[0].legend(handles=[
    plt.Line2D([], [], color=left_color, marker='o', linestyle='', label='Prepaid'),
    plt.Line2D([], [], color=strip_color_defaulted, marker='o', linestyle='', label='Defaulted')
], title='Loan Status', loc='upper right')

# 2. Density plot for both loan statuses
sns.kdeplot(data=eda_train[eda_train['loan_status'] == 0], x='dti', fill=True, alpha=0.6,
            color=right_color_prepaid, label='Prepaid (0)', ax=axes[1])
sns.kdeplot(data=eda_train[eda_train['loan_status'] == 1], x='dti', fill=True, alpha=0.6,
            color=right_color_defaulted, label='Defaulted (1)', ax=axes[1])
axes[1].set_title("DTI Density Plot by Loan Status", fontsize=14, fontweight='bold')
axes[1].set_xlabel("Debt-to-Income Ratio")
axes[1].set_ylabel("Density")
axes[1].legend(title='Loan Status')

# Overall title and layout
plt.suptitle("Modern Warm-Toned Visualization of DTI and Default Risk", fontsize=17, fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



📊 **Insights:**

The Debt-to-Income Ratio (DTI) serves as a crucial indicator of a borrower's financial stress and repayment capacity. From the visual analysis, we observe that defaulted loans tend to cluster around higher DTI levels, especially beyond the 40% threshold. While the difference in median DTI between prepaid (36.0%) and defaulted (40.0%) loans may appear modest, the increase in defaults above this range signals a growing repayment burden as more of the borrower's income is committed to existing debts. This is aligned with industry standards: the **Consumer Financial Protection Bureau (CFPB)** generally considers DTI above 43% as indicative of increased default risk for qualified mortgages.

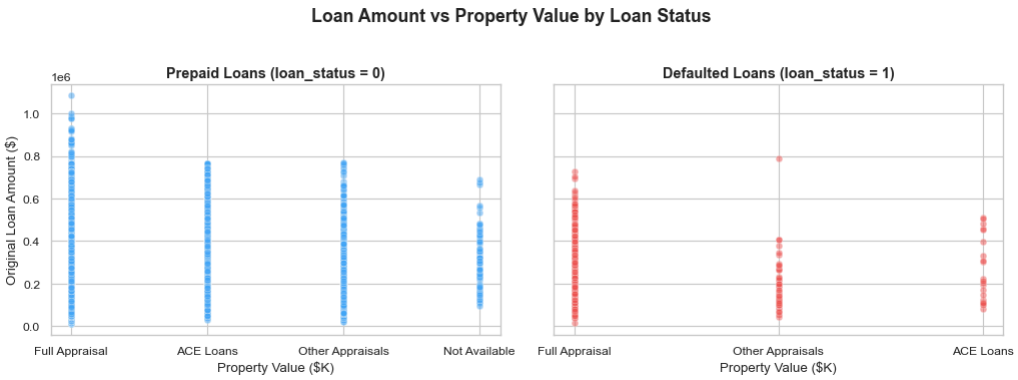
The visualization offers both a micro and macro view of borrower behavior. On the individual level, the granular strip plot reveals that many defaulted loans originate from borrowers with DTI values near the regulatory boundary, reinforcing the need for tighter monitoring of this segment. The stacked histogram, on the other hand, highlights that while most borrowers are within acceptable DTI ranges, there is a noticeable surge in defaults as DTI approaches the 50% mark. This dual perspective allows leadership to assess not just systemic risk, but also uncover patterns among seemingly "normal" borrowers who might still represent a hidden threat to portfolio stability.

Strategically, these insights suggest the value of incorporating DTI thresholds more explicitly into underwriting and monitoring protocols. A possible enhancement would be to create a tiered risk classification system where borrowers with DTI > 43% are subject to additional scrutiny or mitigative strategies such as collateral requirements or income verification audits. These recommendations are supported by academic findings, such as those by **Campbell, Jackson, et al. (2011)**, who note that loan default is highly sensitive to affordability metrics like DTI, especially during periods of economic volatility. By leveraging this evidence-based approach, leadership can enhance portfolio resilience while maintaining regulatory alignment.

4. 💰 Loan Amount vs Property Value by Outcome

Purpose: See how the `orig_upb` (loan amount) compares with `property_val`.

In [85]:
```
# Create side-by-side scatter plots for prepaid and defaulted
fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharey=True)

# Filtered datasets
prepaid_df = eda_train[eda_train['loan_status'] == 0]
defaulted_df = eda_train[eda_train['loan_status'] == 1]

# Prepaid
sns.scatterplot(ax=axes[0], data=prepaid_df,
                x='property_val', y='orig_upb',
                alpha=0.5, color='#42a5f5')
axes[0].set_title("Prepaid Loans (loan_status = 0)", fontsize=13, fontweight='bold')
axes[0].set_xlabel("Property Value ($K)")
axes[0].set_ylabel("Original Loan Amount ($)")
axes[0].grid(True)

# Defaulted
sns.scatterplot(ax=axes[1], data=defaulted_df,
                x='property_val', y='orig_upb',
                alpha=0.5, color='#ef5350')
axes[1].set_title("Defaulted Loans (loan_status = 1)", fontsize=13, fontweight='bold')
axes[1].set_xlabel("Property Value ($K)")
axes[1].set_ylabel("")  # Hide to avoid repetition
axes[1].grid(True)

# Main title and layout
plt.suptitle("Loan Amount vs Property Value by Loan Status", fontsize=16, fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



📊 **Insights:**

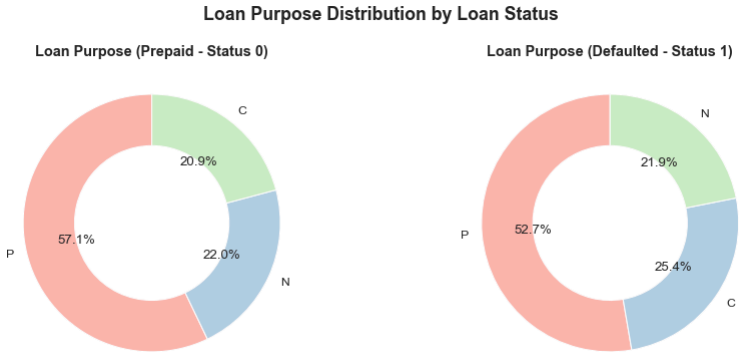
This visualization compares original loan amounts across different property appraisal methods, segmented by loan outcome—prepaid (left) vs. defaulted (right). Prepaid loans span a wide range of amounts and are distributed across all appraisal types, with **Full Appraisal** loans reaching the highest values. In contrast, defaulted loans are concentrated in the **lower loan amount** range, particularly among **ACE Loans** and **Other Appraisals**, suggesting a potential link between lower-value loans and increased risk when traditional appraisal rigor is not applied. Interestingly, the **'Not Available'** appraisal category shows no defaults, which may indicate **incomplete data** or selection bias, raising concerns about model accuracy and data integrity.

From a business standpoint, Freddie Mac can leverage these insights to **refine its credit risk strategies**. Given the visible concentration of defaults in lower-value loans with non-full appraisals, the company may consider **enhancing underwriting criteria** or **risk-based pricing** for loans originated under ACE or similar frameworks. Additionally, ensuring **completeness in appraisal data** will improve model reliability and allow more accurate early warning systems. These steps align with existing research that warns of **valuation risk** in automated or alternative appraisal methods, especially in less standardized markets (Avery et al., 2019; Bhardwaj & Sengupta, 2020).

5. 🏡 Loan Purpose and Default Likelihood

Purpose: To explore which loan purposes are associated with higher default rates.

In [86]:
```
#IF SHORT ON SPACE COULD DELETE

# Create subsets
prepaid = eda_train[eda_train['loan_status'] == 0]['loan_purpose'].value_counts()
defaulted = eda_train[eda_train['loan_status'] == 1]['loan_purpose'].value_counts()

# Color palette
colors = plt.cm.Pastel1.colors

# Plot
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Donut for Prepaid Loans
axes[0].pie(prepaid, labels=prepaid.index, autopct='%1.1f%%', startangle=90,
            colors=colors, wedgeprops={'width': 0.4})
axes[0].set_title('Loan Purpose (Prepaid - Status 0)', fontsize=13, fontweight='bold')

# Donut for Defaulted Loans
axes[1].pie(defaulted, labels=defaulted.index, autopct='%1.1f%%', startangle=90,
            colors=colors, wedgeprops={'width': 0.4})
axes[1].set_title('Loan Purpose (Defaulted - Status 1)', fontsize=13, fontweight='bold')

# Super title and layout
plt.suptitle("Loan Purpose Distribution by Loan Status", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```



Insights:

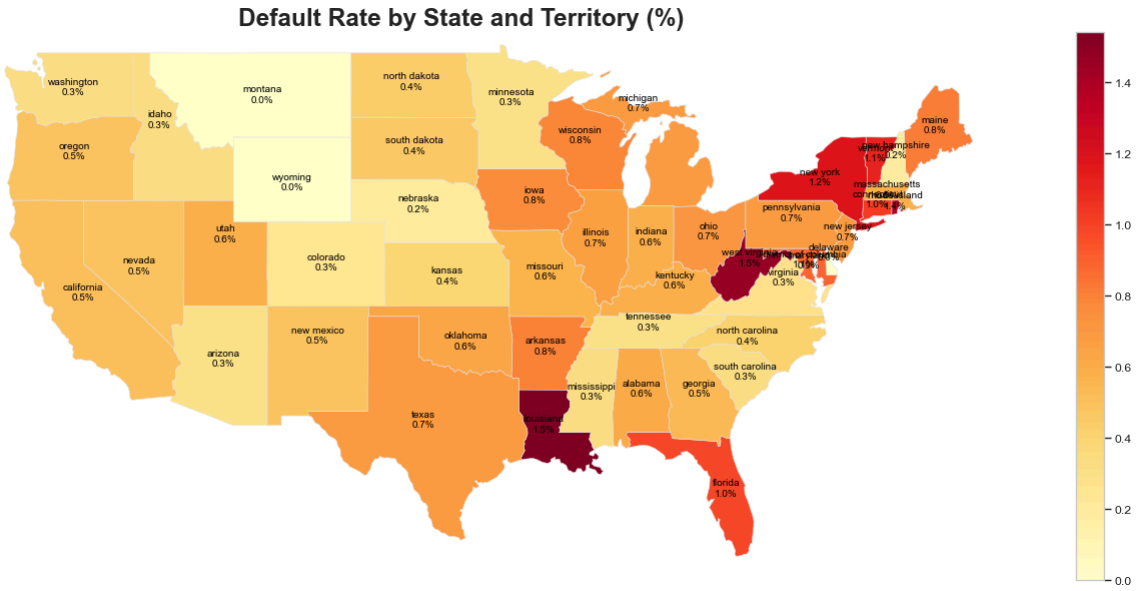
This dual donut chart highlights the distribution of loan purposes among prepaid and defaulted mortgages. Focusing on the defaulted loans (right chart), we observe that while the majority (52.7%) are still for home purchases (label "P"), there is a **notably higher proportion of "C" (Cash-out refinance)** and "N" (No Cash-out refinance) purposes compared to prepaid loans. Specifically, cash-out refinances make up **25.4%** of defaulted loans versus just **20.9%** in prepaid loans. This indicates that borrowers who are extracting equity from their homes—often a sign of financial stress or debt consolidation—may face higher default risks.

This insight suggests a strategic opportunity for Freddie Mac and lending institutions to **enhance scrutiny on refinance-originated loans**, especially those flagged for cash-out purposes. Prior studies have shown that borrowers opting for cash-out refinancing often have **weaker credit profiles** or are more vulnerable to economic shocks due to increased debt burdens (Foote, Gerardi & Willen, 2008). By integrating loan purpose into risk scoring models, lenders can better tailor intervention strategies, pricing models, or counseling programs to preempt default scenarios. Such measures not only reduce credit risk exposure but also support sustainable homeownership.

6. State-wise Loan Distribution

Purpose: Visualize which US states have the most loans in this dataset.

In [87]:
```
import geopandas as gpd
import pandas as pd


# Load US States GeoJSON (from GitHub)
us_states = gpd.read_file("https://raw.githubusercontent.com/PublicaMundi/MappingAPI/master/data/geojson/us-states.json")

# Manual mapping from state names to postal abbreviations (STUSPS)
state_name_to_abbr = {
    'Alabama': 'AL',   'Arizona': 'AZ', 'Arkansas': 'AR', 'California': 'CA',
    'Colorado': 'CO', 'Connecticut': 'CT', 'Delaware': 'DE', 'District of Columbia': 'DC',
    'Florida': 'FL', 'Georgia': 'GA',  'Idaho': 'ID', 'Illinois': 'IL',
    'Indiana': 'IN', 'Iowa': 'IA', 'Kansas': 'KS', 'Kentucky': 'KY', 'Louisiana': 'LA',
    'Maine': 'ME', 'Maryland': 'MD', 'Massachusetts': 'MA', 'Michigan': 'MI', 'Minnesota': 'MN',
    'Mississippi': 'MS', 'Missouri': 'MO', 'Montana': 'MT', 'Nebraska': 'NE', 'Nevada': 'NV',
    'New Hampshire': 'NH', 'New Jersey': 'NJ', 'New Mexico': 'NM', 'New York': 'NY',
    'North Carolina': 'NC', 'North Dakota': 'ND', 'Ohio': 'OH', 'Oklahoma': 'OK',
    'Oregon': 'OR', 'Pennsylvania': 'PA', 'Rhode Island': 'RI', 'South Carolina': 'SC',
    'South Dakota': 'SD', 'Tennessee': 'TN', 'Texas': 'TX', 'Utah': 'UT', 'Vermont': 'VT',
    'Virginia': 'VA', 'Washington': 'WA', 'West Virginia': 'WV', 'Wisconsin': 'WI',
    'Wyoming': 'WY'
}

# Add STUSPS column to the GeoDataFrame
us_states['STUSPS'] = us_states['name'].map(state_name_to_abbr)

# Define valid state codes used in your data
all_states_and_territories = list(state_name_to_abbr.values())

# Optional: filter only matching states
us_states = us_states[us_states['STUSPS'].isin(all_states_and_territories)]

# --- Load or use your loan-level data (already cleaned & filtered) ---
# Assuming eda_train is already loaded and includes 'loan_status' and 'st'

# Calculate default rate per state
default_rate_df = (
    eda_train[eda_train['st'].isin(all_states_and_territories)]
    .groupby('st')['loan_status']
    .agg(total_loans='count', defaults='sum')
    .reset_index()
)

# Compute percentage default rate
default_rate_df['default_rate'] = 100 * default_rate_df['defaults'] / default_rate_df['total_loans']

# Rename 'st' to match merge key
default_rate_df = default_rate_df.rename(columns={'st': 'STUSPS'})

# Merge with GeoDataFrame
merged = us_states.merge(default_rate_df, on='STUSPS', how='left')

# --- Plotting ---
fig, ax = plt.subplots(1, 1, figsize=(18, 7.5))

merged.plot(
    column='default_rate',
    cmap='YlOrRd',
    linewidth=0.5,
    edgecolor='0.9',
    legend=True,
    ax=ax,
    missing_kwds={"color": "lightgrey", "label": "No data"}
)

# Focus on continental US
ax.set_xlim(-130, -65)
ax.set_ylim(24, 50)

# Add percentage and stateslabels
for idx, row in merged.iterrows():
    if not pd.isna(row['default_rate']):
        point = row['geometry'].representative_point()
        label = f"{row['name'].lower()}\n{row['default_rate']:.1f}%"
        ax.text(point.x, point.y, label, ha='center', fontsize=9, color='black')


# Final touches
ax.set_title("Default Rate by State and Territory (%)", fontsize=22, fontweight='bold')
ax.axis('off')
plt.tight_layout()
plt.show()
```



Insights:

The visualizations reveal significant geographic disparities in mortgage default rates across the U.S. States in the Southeast (e.g., Mississippi, Louisiana, Florida, West Virginia) consistently exhibit higher default rates, exceeding 1.5%, while states in the Midwest and West (e.g., Wyoming, Montana, Nebraska) have notably lower rates—some approaching 0%. These trends suggest that regional economic conditions, such as income levels, employment volatility, and housing market stability, may heavily influence borrower risk. Such geographic variation is consistent with findings from previous studies, including Gerardi et al. (2008), which noted the role of local economic shocks in driving default rates.

For Freddie Mac, these geospatial patterns are critical for targeted risk mitigation and portfolio optimization. By integrating state-level default risk into pricing, underwriting, and reserve strategies, Freddie Mac can enhance its responsiveness to local vulnerabilities. This also enables more efficient allocation of resources such as counseling programs or borrower support in regions of concern. Moreover, it supports fairer credit risk modeling that adjusts for structural regional inequalities. Enhanced geospatial intelligence aligns with recommendations in mortgage risk literature for incorporating local indicators to improve predictive power and fairness (Bhutta et al., 2020).

7. 📉 Interest Rates and Loan Status

Purpose: Explore if higher `int_rt` is linked to default risk.

In [88]:
```
plt.figure(figsize=(8,5))
sns.violinplot(x='loan_status', y='int_rt', hue= 'loan_status', data=eda_train, palette='Accent')
plt.title("Interest Rates and Loan Status", fontsize=14)
plt.xlabel("Loan Status")
plt.ylabel("Interest Rate (%)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Insight:

The visualization reveals a notable rightward shift in the interest rate distribution for defaulted loans (loan_status = 1) compared to prepaid loans (loan_status = 0). While prepaid loans tend to cluster tightly around interest rates of 3.5–4.5%, defaulted loans display a wider spread and higher median interest rates, with several cases reaching above 6%. This suggests that borrowers with higher interest rates are more susceptible to default, possibly due to greater monthly payment burdens and weaker creditworthiness at origination. These observations align with previous research indicating that higher interest burdens can exacerbate repayment difficulty—especially in volatile economic periods (Foote, Gerardi & Willen, 2008).

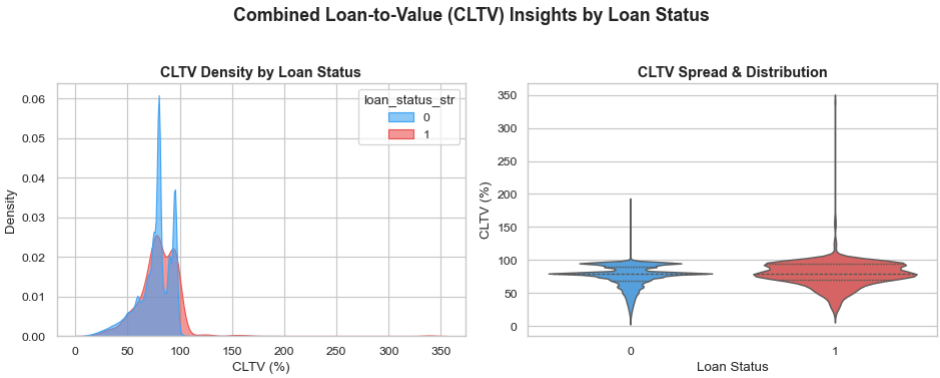
For Freddie Mac, these findings highlight the predictive value of interest rate as a key feature in default risk modeling. Loans with above-average rates may warrant enhanced monitoring, tailored mitigation strategies (e.g. refinancing offers or early risk flags), or conservative pricing in secondary market sales. Additionally, this insight strengthens the case for offering affordable mortgage options and rate-reduction incentives as a means of minimizing default incidence—thereby protecting institutional portfolios and borrowers alike.

8. CLTV Insights: Understanding Risk Patterns Through Loan-to-Value Distribution

Purpose: to explore how the Combined Loan-to-Value (CLTV) ratio differs between defaulted (1) and prepaid (0) loans. It helps identify potential patterns or risk signals in CLTV that are associated with different loan outcomes.

In [89]:
```
# Convert loan_status to string for palette compatibility
eda_train['loan_status_str'] = eda_train['loan_status'].astype(str)

# Set Seaborn style
sns.set(style='whitegrid')

# Define a custom colorful palette with string keys
palette = {'0': '#42a5f5', '1': '#ef5350'}  # blue for prepaid, red for defaulted

# Create figure with 3 subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# 1. KDE Density Plot
sns.kdeplot(
    data=eda_train,
    x='cltv',
    hue='loan_status_str',
    fill=True,
    common_norm=False,
    palette=palette,
    alpha=0.6,
    ax=axes[0]
)
```

```
axes[0].set_title('CLTV Density by Loan Status', fontsize=13, fontweight='bold')
axes[0].set_xlabel('CLTV (%)')
axes[0].set_ylabel('Density')

# 3. Violin Plot
sns.violinplot(
    data=eda_train,
    x='loan_status_str',
    y='cltv',
    hue='loan_status_str',
    palette=palette,
    inner='quart',
    ax=axes[1]
)
axes[1].set_title('CLTV Spread & Distribution', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Loan Status')
axes[1].set_ylabel('CLTV (%)')

# Overall figure title
plt.suptitle("Combined Loan-to-Value (CLTV) Insights by Loan Status", fontsize=16, fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



📊 **Insight:**

The three-panel plot clearly illustrates that borrowers who defaulted (loan_status = 1) tend to exhibit higher CLTV ratios compared to those who prepaid (loan_status = 0). The density plot on the left shows a rightward shift for defaulted loans, indicating that their loan amounts were often a higher proportion of the total appraised property value. The boxplot in the center supports this, with the median CLTV for defaulted loans being slightly higher and displaying a broader interquartile range, suggesting greater variability and exposure to risk. Additionally, the violin plot on the right reveals a thicker tail and more concentration around higher CLTV values among defaulted loans.

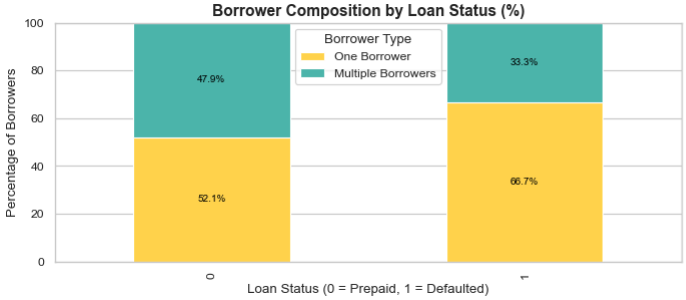
For Freddie Mac, this emphasizes the predictive significance of CLTV in mortgage default risk modeling. High CLTV ratios reflect lower borrower equity at origination, making defaults more likely, especially during economic downturns or home price depreciation. Incorporating CLTV thresholds into underwriting policies, dynamic pricing, or early-warning systems can enhance Freddie Mac's ability to mitigate losses and better assess the capital reserves needed for riskier loan segments. These findings align with prior literature noting that borrowers with minimal equity are more likely to default (Gerardi et al., 2013).

9. 👥 Impact of Number of Borrowers on Default Risk

Purpose: Investigate whether having multiple borrowers reduces risk.

In [90]:
```
borrower_counts = eda_train.groupby(['cnt_borr', 'loan_status']).size().unstack(fill_value=0)
borrower_percent = borrower_counts.div(borrower_counts.sum(axis=1), axis=0) * 100
borrower_percent_rounded = borrower_percent.round(1).astype(str) + '%'
# Assume this is your preprocessed count table
# Index: Borrower Type, Columns: Loan Status (0 = prepaid, 1 = defaulted)
borrower_counts = pd.DataFrame(borrower_counts, index=['One Borrower', 'Multiple Borrowers'])

# 1. Compute percentages (column-wise: per loan status)
borrower_percents = borrower_counts.div(borrower_counts.sum(axis=0), axis=1) * 100

# 2. Transpose for plotting (loan status becomes index)
plot_percent = borrower_percents.T
plot_count = borrower_counts.T

# 3. Ensure consistent column order
order = ['One Borrower', 'Multiple Borrowers']
plot_percent = plot_percent.reindex(columns=order)
plot_count = plot_count.reindex(columns=order)

# 4. Build table text: count + percent
table_combined = plot_count.astype(str) + ' (' + plot_percent.round(1).astype(str) + '%)'

# === Plotting ===
fig, axes = plt.subplots(1, 2, figsize=(14, 4), gridspec_kw={'width_ratios': [2.5, 1]})

# LEFT: Stacked % bar chart
bars = plot_percent.plot(
    kind='bar',
    stacked=True,
    ax=axes[0],
    color={'One Borrower': '#ffd54f', 'Multiple Borrowers': '#4db6ac'}
)
axes[0].set_title("Borrower Composition by Loan Status (%)", fontsize=14, fontweight='bold')
axes[0].set_xlabel("Loan Status (0 = Prepaid, 1 = Defaulted)")
axes[0].set_ylabel("Percentage of Borrowers")
axes[0].legend(title="Borrower Type", loc='upper center')
axes[0].set_ylim(0, 100)

# Annotate percentage inside bars
for container in bars.containers:
    for bar in container:
        height = bar.get_height()
        if height > 2:
            axes[0].text(
                bar.get_x() + bar.get_width()/2,
                bar.get_y() + height/2,
                f'{height:.1f}%',
                ha='center',
                va='center',
                fontsize=9,
                color='black'
            )

# RIGHT: Fancy Table with counts and %
axes[1].axis('off')
table = axes[1].table(
    cellText=table_combined.values,
    rowLabels=['Prepaid (0)', 'Defaulted (1)'],
    colLabels=table_combined.columns,
    cellLoc='center',
    loc='center'
)

# Styling
```

```
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1.3, 1.5)

# Header and stripe styling
for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_facecolor('#424242')
        cell.set_text_props(weight='bold', color='white')
    elif row % 2 == 0:
        cell.set_facecolor('#f7f7f7')
    else:
        cell.set_facecolor('#ffffff')

axes[1].set_title("Count + % by Loan Status", fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()
```



Count + % by Loan Status

	One Borrower	Multiple Borrowers
Prepaid (0)	52455 (52.1%)	48294 (47.9%)
Defaulted (1)	396 (66.7%)	198 (33.3%)

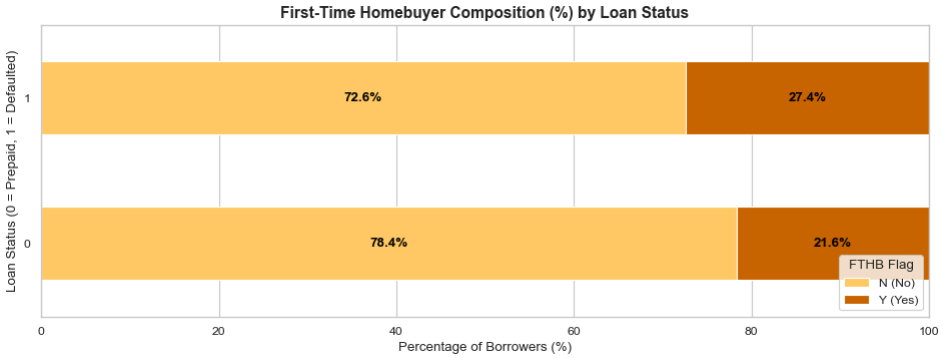
The visualization illustrates the composition of borrowers by loan status, revealing that among defaulted loans (loan_status = 1), a significant **66.7% were associated with single borrowers**, compared to just **33.3% for multiple borrowers**. In contrast, prepaid loans (loan_status = 0) show a more balanced distribution between borrower types—52.1% single and 47.9% multiple. This pattern strengthens the case that loans with only one borrower are inherently more vulnerable to default. Having a second borrower may act as a financial buffer, especially in times of economic hardship, by diversifying income sources and repayment responsibility.

For Freddie Mac and mortgage stakeholders, this insight holds actionable implications. Incorporating borrower type as a **predictive feature in default risk modeling** could improve the institution's ability to identify high-risk applicants early. For example, offering additional financial counseling or structuring more conservative loan terms for single-borrower applications might mitigate risks. This aligns with studies such as Quercia et al. (2012), which emphasize the importance of **household structure and shared liability** in mortgage performance. Proactively adjusting risk strategies based on this factor can strengthen portfolio stability and enhance long-term lending outcomes.

10. 🔍 Are First-Time Homebuyers More Likely to Default?

Purpose: See if `flag_fthb` (first-time homebuyer) is linked to higher default rates.

In [91]:
```
# Hitung jumlah berdasarkan loan_status dan FTHB flag
fthb_counts = eda_train.groupby(['loan_status', 'flag_fthb']).size().unstack(fill_value=0)

# Konversi ke persentase
fthb_percent = fthb_counts.div(fthb_counts.sum(axis=1), axis=0) * 100

# Plot horizontal stacked bar chart dengan persentase
colors = ['#ffcc66', '#cc6600']  # N = light orange, Y = darker orange
fthb_percent.plot(kind='barh', stacked=True, color=colors, figsize=(12, 4.7))

# Tambahkan anotasi persentase pada masing-masing bagian bar
for idx, (status, row) in enumerate(fthb_percent.iterrows()):
    xpos = 0
    for label, value in row.items():
        if value > 3:  # tampilkan label hanya jika cukup besar untuk keterbacaan
            plt.text(xpos + value/2, idx, f"{value:.1f}%", ha='center', va='center', color='black', fontweight='bold')
        xpos += value

# Ganti label dan style axis
plt.xlabel("Percentage of Borrowers (%)")
plt.ylabel("Loan Status (0 = Prepaid, 1 = Defaulted)")
plt.title("First-Time Homebuyer Composition (%) by Loan Status", fontsize=14, fontweight='bold')
plt.xlim(0, 100)
plt.legend(title='FTHB Flag', labels=['N (No)', 'Y (Yes)'], loc='lower right')

plt.tight_layout()
plt.show()
```



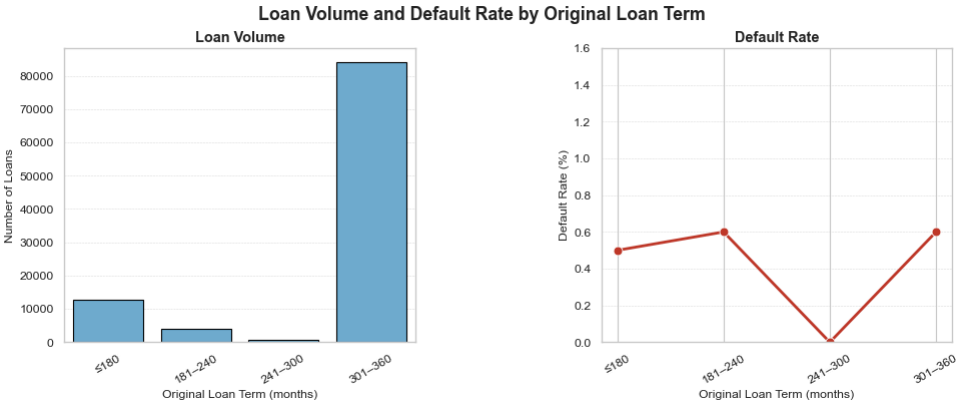
The visualization showcases the composition of first-time homebuyers (FTHB) across different loan statuses. It reveals that among borrowers who defaulted (loan_status = 1), **27.4% were first-time homebuyers**, which is higher than the **21.6% among prepaid loans** (loan_status = 0). This suggests that first-time homebuyers are overrepresented among defaulted loans, indicating a **potential vulnerability for this segment** of borrowers. Given their limited experience in managing mortgage payments and possibly lower financial literacy, this group may face greater challenges in sustaining long-term repayment.

For Freddie Mac, this insight is vital when designing **risk mitigation and support strategies** for new homeowners. Targeted financial education, personalized risk-based underwriting, or offering more conservative loan products to first-time buyers could help reduce the likelihood of default. These findings are consistent with prior literature, such as Bhutta & Hizmo (2021), which showed that **first-time borrowers exhibit higher default probabilities**, even after controlling for credit score and loan type. By integrating FTHB indicators into risk scoring models, Freddie Mac can enhance **portfolio resilience** and support **sustainable homeownership**.

11. 📊 Default Rate and Loan Volume by Original Loan Term

In [92]:
```
# Original loan terms up to a maximum of 360 months (max value in train data)
bins = [0, 180, 240, 300, 360.1]
labels = ["≤180", "181–240", "241–300", "301–360"]
eda_train["term_bin"] = pd.cut(eda_train["orig_loan_term"], bins=bins, labels=labels, include_lowest=True)

# Group by bin and compute default rate for each bin
term_stats = (
    eda_train.groupby("term_bin", observed=False)["loan_status"]
    .agg(count="count", default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index()
)
term_stats["default_rate"] = term_stats["default_rate"].round(1)  # round for cleaner labels
```

```
# Two plots side-by-side
fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(12, 5),
    sharey=False,
    gridspec_kw={'wspace': 0.3},
    constrained_layout=True
)

# Left: Volume in each bin
sns.barplot(
    data=term_stats, x="term_bin", y="count", ax=ax1,
    color="#5dade2", edgecolor="black"
)
ax1.set_title("Loan Volume", fontsize=13, fontweight="bold")
ax1.set_xlabel("Original Loan Term (months)", fontsize=11)
ax1.set_ylabel("Number of Loans", fontsize=11)
ax1.tick_params(axis='x', rotation=30)
ax1.grid(True, axis='y', linestyle='--', linewidth=0.5, alpha=0.6)

# Right: Default rate per term bin
sns.lineplot(
    data=term_stats, x="term_bin", y="default_rate", ax=ax2,
    color="#c0392b", marker='o', linewidth=2.5, markersize=8
)
ax2.set_title("Default Rate", fontsize=13, fontweight="bold")
ax2.set_xlabel("Original Loan Term (months)", fontsize=11)
ax2.set_ylabel("Default Rate (%)", fontsize=11)
ax2.set_ylim(0, term_stats["default_rate"].max() + 1)
ax2.tick_params(axis='x', rotation=30)
ax2.grid(True, axis='y', linestyle='--', linewidth=0.5, alpha=0.6)

# Overall title
fig.suptitle("Loan Volume and Default Rate by Original Loan Term", fontsize=16, fontweight="bold")

plt.show()
```



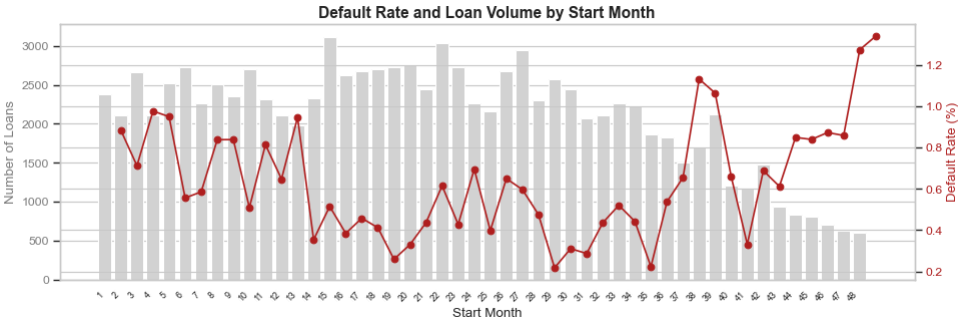
Insights:

- The default rate does not appear to be substantially influenced by the original loan term. While there is a visible drop in the 241–300 months bin, this is because of the extremely low number of loans issued within that range, making the observed rate statistically unreliable.
- Across the other bins, default rates remain relatively stable, even in the presence of large differences in loan volume. This suggests that loan duration, by itself, may not be a strong predictor of default risk in this dataset.

📌 **Observation**: The majority of loans fall within the 301–360 months range, indicating a strong preference—either from borrowers or institutional policy—for longer-term mortgages (i.e., 25–30 years).

12. 🕒 Default Rate and Loan Volume Over Time by Start Month

In [93]:
```
# Group loan count and default rate by start month
start_stats = (
    eda_train.groupby("start_months", observed=False)["loan_status"]
    .agg(loan_count="count", default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index()
    .sort_values("start_months")
)

# Drop final months with very low number of loans
start_stats = start_stats[start_stats["loan_count"] > 50]

# Plotting
fig, ax1 = plt.subplots(figsize=(12, 4), constrained_layout=True)

# Barplot: loan volume
sns.barplot(data=start_stats, x="start_months", y="loan_count", ax=ax1, color="lightgray")
ax1.set_ylabel("Number of Loans", color="gray")
ax1.set_xlabel("Start Month")
ax1.tick_params(axis='y', labelcolor='gray')
plt.setp(ax1.get_xticklabels(), rotation=45, ha='right', fontsize=8)


# Lineplot: default rate using twin axis
ax2 = ax1.twinx()
ax2.plot(start_stats["start_months"], start_stats["default_rate"], color="firebrick", marker="o", linewidth=1.5)
ax2.set_ylabel("Default Rate (%)", color="firebrick")
ax2.tick_params(axis='y', labelcolor='firebrick')

# Title
ax1.set_title("Default Rate and Loan Volume by Start Month", fontsize=14, fontweight="bold")

plt.show()
```



The plot above illustrates how both the default rate (%) and the number of issued loans change over time, based on the loan's start month.

📌 **Key Observations**:

- The **default rate** (red line) does **not remain constant**: it starts relatively high, then decreases and remains low for a central range of months, before strongly rising again in the most recent months.
- The **loan volume** (grey bars) is relatively stable in the early period, but shows a **notable drop** toward the end of the timeline.

Low default rates in the middle period may reflect:

- Favorable economic conditions
- Stricter credit policies during that time frame.

Increasing default rate in recent months could be influenced by:

- A shift toward riskier lending or economic stressors (e.g., inflation, post-pandemic recovery)
- A **statistical distortion**, as many recent loans are likely still active

⚠️ **Note**: The recent uptick in defaults, despite lower loan issuance, should be interpreted with caution due to limited observation time for these loans.

13. 🏦 Default Rate Analysis by Loan Servicer

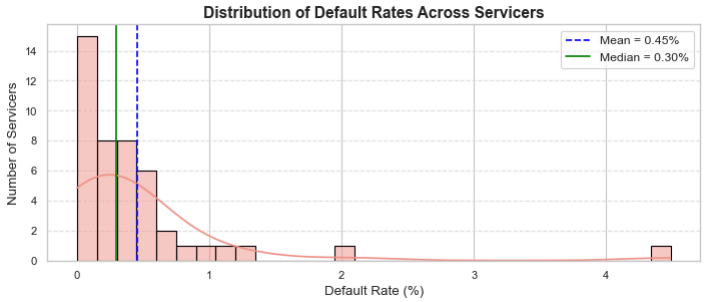
In [94]:
```
#default rate per servicer
servicer_stats = (eda_train.groupby("servicer_name", observed=False)["loan_status"]
                  .agg(total_loans="count", default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index().sort_values("default_rate", ascending=False))

#summary statistics as table
servicer_summary = servicer_stats["default_rate"].describe(percentiles=[0.25, 0.5, 0.75])
servicer_summary_df = servicer_summary.to_frame(name="default_rate (%)")
print("Summary Statistics for Default Rates Across Servicers:")
display(servicer_summary_df)

# top and worse servicers by default rate
top_10 = servicer_stats.head(10).reset_index(drop=True)
bottom_10 = servicer_stats.tail(10).reset_index(drop=True)

top_10.columns = [f"Top_{col}" for col in top_10.columns]
bottom_10.columns = [f"Bottom_{col}" for col in bottom_10.columns]

combined_servicer_table = pd.concat([top_10, bottom_10], axis=1)

# Display
print("Servicer Default Rate Summary + Top & Bottom 10 Servicers (Horizontal View):")
display(combined_servicer_table)

#hist
mean_rate = servicer_stats["default_rate"].mean()
median_rate = servicer_stats["default_rate"].median()

plt.figure(figsize=(9, 4))
sns.histplot(data=servicer_stats, x="default_rate", bins=30,
             kde=True, color="#f1948a",edgecolor="black")
#mean
plt.axvline(mean_rate, color='blue', linestyle='--',
            linewidth=1.5, label=f"Mean = {mean_rate:.2f}%")
#median
plt.axvline(median_rate, color='green', linestyle='-',
            linewidth=1.5, label=f"Median = {median_rate:.2f}%")

plt.title("Distribution of Default Rates Across Servicers", fontsize=14, fontweight='bold')
plt.xlabel("Default Rate (%)", fontsize=12)
plt.ylabel("Number of Servicers", fontsize=12)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

Summary Statistics for Default Rates Across Servicers:

	default_rate (%)
count	45.000000
mean	0.452581
std	0.739128
min	0.000000
25%	0.000000
50%	0.297619
75%	0.510204
max	4.486346

Servicer Default Rate Summary + Top & Bottom 10 Servicers (Horizontal View):

	Top_servicer_name	Top_total_loans	Top_default_rate	Bottom_servicer_name	Bottom_total_loans	Bottom_default_rate
0	SPECIALIZED LOAN SERVICING LLC	1538	4.486346	BANK OF AMERICA, N.A.	341	0.0
1	MARLIN MORTGAGE CAPITAL, LLC	48	2.083333	CMG MORTGAGE, INC.	108	0.0
2	FLAGSTAR BANK, FSB	78	1.282051	FINANCE OF AMERICA MORTGAGE LLC	43	0.0
3	COLONIAL SAVINGS, F.A.	84	1.190476	CROSSCOUNTRY MORTGAGE, LLC	22	0.0
4	Other servicers	27491	0.993052	GUARANTEED RATE, INC.	73	0.0
5	ROCKET MORTGAGE, LLC	2209	0.814848	ONSLOW BAY FINANCIAL LLC	91	0.0
6	NATIONSTAR MORTGAGE LLC DBA MR. COOPER	2451	0.652795	PENNYMAC LOAN SERVICES, LLC	18	0.0
7	FIFTH THIRD BANK, NATIONAL ASSOCIATION	933	0.643087	PODIUM MORTGAGE CAPITAL LLC	120	0.0
8	U.S. BANK N.A.	3899	0.564247	ROUNDPOINT MORTGAGE SERVICING CORPORATION	98	0.0
9	LOANDEPOT.COM, LLC	717	0.557880	SELECT PORTFOLIO SERVICING, INC.	175	0.0



The distribution of default rates across servicers is heavily **right-skewed**, with most institutions exhibiting very low default rates.

📊 **Summary statistics**:

- The **mean** default rate is **0.45%**, but the **median** is lower at **0.30%**, indicating that a few servicers with very high default rates are skewing the average.
- At least **25%** of servicers had a 0% default rate, and **75%** were below **0.51%**.

🔴 **Worst 10 servicers** show significantly elevated default rates, possibly due to:

- Riskier borrowers
- Inefficient loan servicing or management
- Smaller loan counts amplifying the impact of individual defaults
- Especially SPECIALIZED LOAN SERVICING LLC, with 1538 loans and 4.49 % rate default should be kept track of

🟢 **Best 10 servicers** all reported **0% default**, which may reflect:

- High-quality service
- Conservative strategies
- In some cases, **limited sample sizes** (small number of loans), which may not provide a representative performance measure.

📈 **Distribution insights**:

- The histogram confirms that most servicers maintain low default rates.
- A small number of outliers reach rates above 1%, with the worst reaching **over 4%**, suggesting major disparities in performance.

⚠️ **Caution**: Interpretation should take into account the number of loans serviced. High or low default rates from institutions with very few loans may not be statistically meaningful.

14. 📄 Default Rate Analysis by Seller

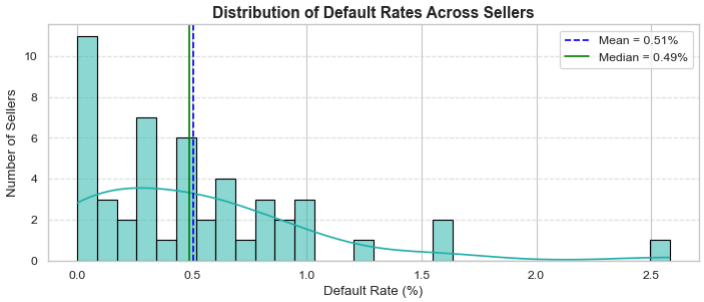
In [95]:
```
#default rate per seller
seller_stats = (eda_train.groupby("seller_name", observed=False)["loan_status"]
    .agg(total_loans="count",default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index().sort_values("default_rate", ascending=False))

#summary statistics per seller
seller_summary = seller_stats["default_rate"].describe(percentiles=[0.25, 0.5, 0.75])
seller_summary_df = seller_summary.to_frame(name="default_rate (%)")
print("Summary Statistics for Default Rates Across Sellers:")
display(seller_summary_df)

# top and bottom 10 sellers by default rate
top_10_sellers = seller_stats.head(10)
bottom_10_sellers = seller_stats.tail(10)

#top and worse 10 sellers
combined_sellers = pd.concat(
    [top_10_sellers.reset_index(drop=True), bottom_10_sellers.reset_index(drop=True)],
    axis=1
)

combined_sellers.columns = [
    "Top Seller", "Top Total Loans", "Top Default Rate (%)",
    "Bottom Seller", "Bottom Total Loans", "Bottom Default Rate (%)"
]

print("omparison of Top 10 and Bottom 10 Sellers by Default Rate:")
display(combined_sellers)

#distribution plot
mean_rate_seller = seller_stats["default_rate"].mean()
median_rate_seller = seller_stats["default_rate"].median()

plt.figure(figsize=(9, 4))
sns.histplot(data=seller_stats, x="default_rate", bins=30,
    kde=True, color="lightseagreen", edgecolor="black")

#mean and media
plt.axvline(mean_rate_seller, color='blue', linestyle='--', linewidth=1.5,
            label=f"Mean = {mean_rate_seller:.2f}%")
plt.axvline(median_rate_seller, color='green', linestyle='-', linewidth=1.5,
            label=f"Median = {median_rate_seller:.2f}%")

#title
plt.title("Distribution of Default Rates Across Sellers", fontsize=14, fontweight='bold')
plt.xlabel("Default Rate (%)")
plt.ylabel("Number of Sellers")
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

```
Summary Statistics for Default Rates Across Sellers:
```

	default_rate (%)
count	49.000000
mean	0.506777
std	0.507106
min	0.000000
25%	0.129870
50%	0.488557
75%	0.706714
max	2.580645

```
omparison of Top 10 and Bottom 10 Sellers by Default Rate:
```

	Top Seller	Top Total Loans	Top Default Rate (%)	Bottom Seller	Bottom Total Loans	Bottom Default Rate (%)
0	NATIONSTAR MORTGAGE LLC	310	2.580645	CITIMORTGAGE, INC.	64	0.0
1	USAA FEDERAL SAVINGS BANK	316	1.582278	CMG MORTGAGE, INC.	118	0.0
2	FIFTH THIRD MORTGAGE COMPANY	192	1.562500	CROSSCOUNTRY MORTGAGE, LLC	22	0.0
3	FIFTH THIRD BANK	82	1.219512	FIFTH THIRD BANK, NATIONAL ASSOCIATION	104	0.0
4	UNITED WHOLESALE MORTGAGE, LLC	97	1.030928	NORTHPOINTE BANK	100	0.0
5	JPMORGAN CHASE BANK, NATIONAL ASSOCIATION	3801	1.026046	LAKEVIEW LOAN SERVICING, LLC	271	0.0
6	HOME POINT FINANCIAL CORPORATION	697	1.004304	PRIMELENDING, A PLAINSCAPITAL COMPANY	352	0.0
7	NATIONSTAR MORTGAGE LLC DBA MR. COOPER	1426	0.911641	PENNYMAC LOAN SERVICES, LLC	18	0.0
8	LOANDEPOT.COM, LLC	2112	0.899621	TEXAS CAPITAL BANK	377	0.0
9	STEARNS LENDING, LLC	730	0.821918	STEARNS LENDING, LLC.	248	0.0



The distribution of default rates across loan sellers is quite skewed to the right, indicating that most sellers maintain relatively low default rates, while a few exhibit significantly higher values.

📊 **Summary statistics**:

- The **mean default rate** is **0.51%**, slightly above the **median** of **0.49%**, suggesting a relatively symmetric distribution with a light tail.
- The **minimum rate** is 0%, and the **maximum** reaches **2.58%**, showing high variability in performance.

🔴 **Worst 10 sellers** by default rate include institutions with both moderate and high loan volumes, implying that poor performance is not necessarily tied to low data counts.

🟢 **Best 10 sellers**, all with 0% default, could reflect:

- High-quality serice
- Strict borrower selection,
- Small sample sizes where no defaults occurred.

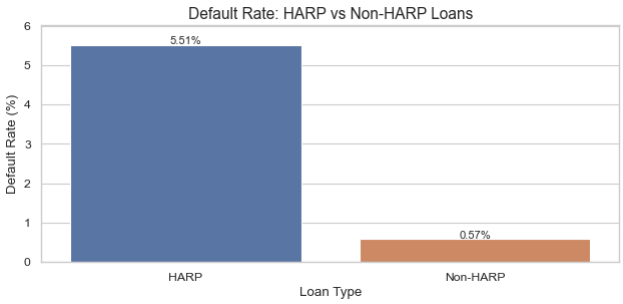
📈 **Distribution insights**:

- The histogram reveals that many sellers have a default rate below 1%, with several reporting **0% default**.
- A small group of sellers has default rates exceeding 1%, which may indicate:
  - Exposure to riskier borrower
  - Or less effective service

⚠️ **Note**: As with servicers, some sellers in the "best" group handle fewer loans (< 100), and their 0% default rates should be interpreted with caution. Larger sample sizes give more reliable estimates of performance.

15. 🆚 Default Rate Comparison: HARP vs Non-HARP Loans

In [96]:
```
# Create a binary label for HARP status
eda_train["is_harp"] = eda_train["harp_loan"].map({'Y': 'HARP', 'N': 'Non-HARP'})

# Group and calculate default rates
harp_comparison = (eda_train.groupby("is_harp")["loan_status"]
                   .agg(default_rate=lambda x: (x == 1).mean() * 100).reset_index())

# Define custom colors: first bar (Non-HARP), second bar (HARP)
custom_colors = ["lightcoral", "lightblue"]  # first = Non-HARP, second = HARP

# Plot
plt.figure(figsize=(8,4))
sns.barplot(data=harp_comparison, x="is_harp", y="default_rate", hue='is_harp',legend=False)

# Add annotations
for index, row in harp_comparison.iterrows():
    plt.text(index, row.default_rate + 0.02, f"{row.default_rate:.2f}%", ha='center', fontsize=10)

plt.title("Default Rate: HARP vs Non-HARP Loans", fontsize=14)
plt.xlabel("Loan Type")
plt.ylabel("Default Rate (%)")
plt.ylim(0, harp_comparison["default_rate"].max() + 0.5)
plt.tight_layout()
plt.show()
```



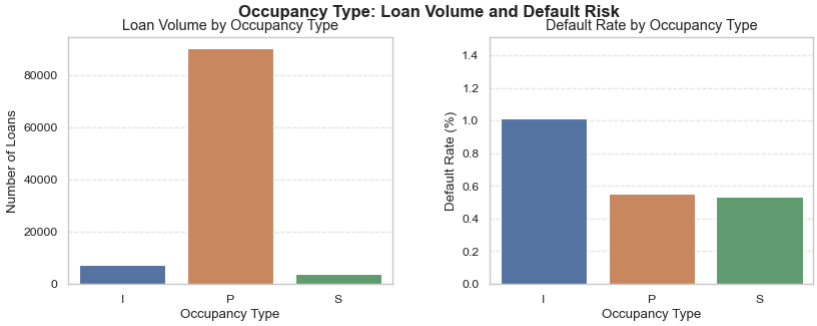
This chart compares the default rates between HARP loans and Non-HARP loans.

📊 **Key Observation**:

- Loans categorized under the **HARP (Home Affordable Refinance Program)** exhibit a significantly higher default rate (**5.51%**) compared to Non-HARP loans (**0.57%**).
- HARP loans are issued to borrowers with **CLTV** (>80) and limited refinancing options, which implies a higher credit risk.
- The difference in default rate suggests that **HARP status is a strong predictive feature** for mortgage default classification.

16. 🏠 Occupancy Type: Loan Volume and Default Risk

In [97]:
```
#default rate per occupacy status
occ_stats = (eda_train.groupby("occpy_sts", observed=False)["loan_status"]
    .agg(loan_count="count",default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index().sort_values("occpy_sts"))

custom_palette = ["#C44E52", "#4C72B0", "#55A868"]  # I, P, S

#subplot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharex=True, gridspec_kw={'wspace': 0.3})

#barplot for volume
sns.barplot(data=occ_stats, x="occpy_sts", y="loan_count", hue="occpy_sts", legend=False, ax=ax1)
ax1.set_title("Loan Volume by Occupancy Type", fontsize=13)
ax1.set_xlabel("Occupancy Type")
ax1.set_ylabel("Number of Loans")
ax1.grid(axis='y', linestyle='--', alpha=0.4)

#barplot per default rate
sns.barplot(data=occ_stats, x="occpy_sts", y="default_rate", hue="occpy_sts", legend=False, ax=ax2)
ax2.set_title("Default Rate by Occupancy Type", fontsize=13)
ax2.set_xlabel("Occupancy Type")
ax2.set_ylabel("Default Rate (%)")
ax2.grid(axis='y', linestyle='--', alpha=0.4)
ax2.set_ylim(0, occ_stats["default_rate"].max() + 0.5)

# Titolo generale
fig.suptitle("Occupancy Type: Loan Volume and Default Risk", fontsize=15, fontweight='bold')
plt.show()
```



This figure compares the number of loans and the default rate (%) across three occupancy types:

- **I** = Investment property
- **P** = Primary residence (owner-occupied)
- **S** = Second home

Loan volume:

- The vast majority of loans are associated with **primary residences (P)**.
- **Investment properties (I)** and **second homes (S)** represent a much smaller share of the portfolio.

Default rate:

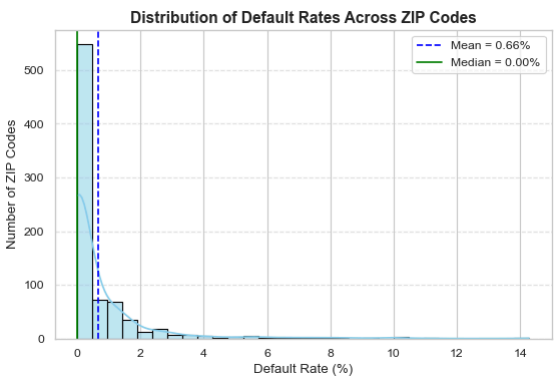
- **Investment properties (I)** show the **highest default rate (~1%)**, likely due to their more speculative nature and borrowers' lower attachment compared to owner-occupied homes.
- **Primary residences (P)** have a moderate default rate (~0.55%), despite being the most common.
- **Second homes (S)** exhibit the **lowest default risk (~0.50%)**, possibly reflecting borrowers that are more financially stable.

Interpretation:

- Occupancy type is a relevant categorical feature for default prediction, showing clear differences in risk profiles.

17. 🗺️ Default Rate Distribution by ZIP Code

In [98]:
```
#loan count and default rate per zipcoded
zip_stats = (eda_train.groupby("zipcode", observed=False)["loan_status"]
    .agg(loan_count="count",default_rate=lambda x: (x == 1).mean() * 100)
    .reset_index().sort_values("default_rate", ascending=False))

#mean and median
mean_zip_rate = zip_stats["default_rate"].mean()
median_zip_rate = zip_stats["default_rate"].median()

#plot
plt.figure(figsize=(8, 5))
sns.histplot(data=zip_stats,x="default_rate",bins=30,
    kde=True,color="skyblue", edgecolor="black")

#mean and median
plt.axvline(mean_zip_rate, color='blue', linestyle='--', linewidth=1.5, label=f"Mean = {mean_zip_rate:.2f}%")
plt.axvline(median_zip_rate, color='green', linestyle='-', linewidth=1.5, label=f"Median = {median_zip_rate:.2f}%")

plt.title("Distribution of Default Rates Across ZIP Codes", fontsize=14, fontweight='bold')
plt.xlabel("Default Rate (%)")
plt.ylabel("Number of ZIP Codes")
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```



This chart illustrates the distribution of mortgage default rates across ZIP codes.The dataset contains approximately **800 unique ZIP codes**.

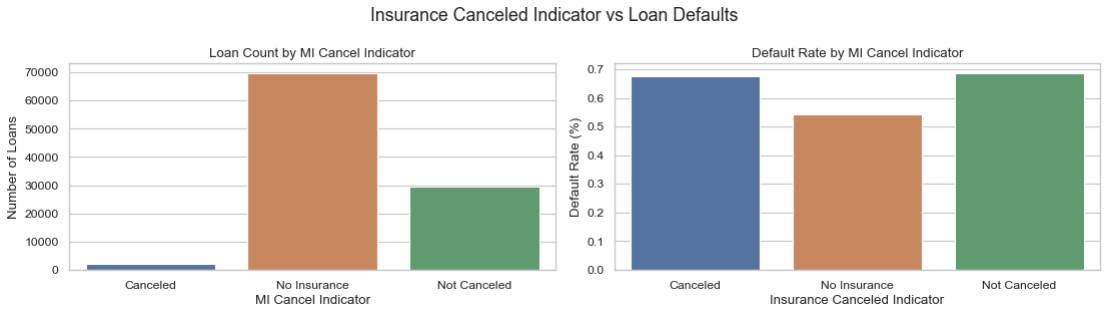
📊 **Key observations**:

- The distribution is **heavily right-skewed**, with the vast majority of ZIP codes exhibiting a **default rate close to 0%**.
- Only a small fraction exceed **2–3%**, and very few surpass **5%**, indicating **localized spikes in default rate**.
- Most geographic areas maintain **very low default risk**, in line with the overall default rate observed in the dataset.
- A few ZIP codes show **concentrated credit risk**, potentially due to **localized economic stress**
- ZIP code appears to be a **powerful spatial feature**: it ffers a more detailed view of local market behavior compared to broader geographic variables like state.

This suggests that including ZIP code could enhance the model's ability to detect risk patterns at the community level.

18. 🛡️ Mortgage Insurance Cancellation and Default Risk

In [99]:
```
#Loan count and default rate per mi_cancel_ind
mi_cancel_stats = (eda_train.groupby("mi_cancel_ind")["loan_status"].agg(
                    loan_count="count",default_rate=lambda x: (x == 1).mean() * 100)
                   .reset_index().sort_values("mi_cancel_ind"))
#palette for each category
categories = mi_cancel_stats["mi_cancel_ind"].astype(str)
palette = dict(zip(categories, sns.color_palette("Set2", n_colors=len(categories))))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))

#Loan volume
sns.barplot(data=mi_cancel_stats, x="mi_cancel_ind", y="loan_count",
            hue = 'mi_cancel_ind',legend = False,ax=ax1)
ax1.set_title("Loan Count by MI Cancel Indicator")
ax1.set_xlabel("MI Cancel Indicator")
ax1.set_ylabel("Number of Loans")

#default rate
sns.barplot(data=mi_cancel_stats, x="mi_cancel_ind", y="default_rate",
            hue = 'mi_cancel_ind',legend = False, ax=ax2)
ax2.set_title("Default Rate by MI Cancel Indicator")
ax2.set_xlabel("Insurance Canceled Indicator")
ax2.set_ylabel("Default Rate (%)")

plt.suptitle("Insurance Canceled Indicator vs Loan Defaults", fontsize=16)
plt.tight_layout()
plt.show()
```



📊 **Loan Volume**:

- The majority of loans fall under the **"No Insurance"** category.
- A smaller portion has **active insurance** ("Not Canceled"), while very few have had their insurance **canceled**.

📈 **Default Rates**:

- Loans with **canceled MI** and those with **active MI** show **similar and relatively high default rates** (~0.68%).
- Loans without insurance present a **slightly lower default rate** (~0.55%).

🔍 **Interpretation**:

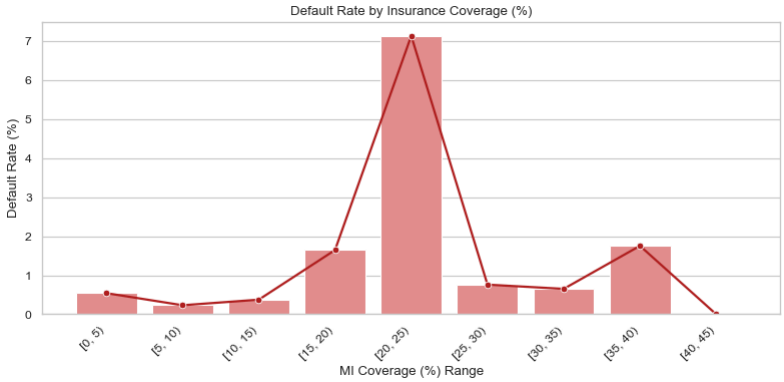
- The similar default risk between "Canceled" and "Not Canceled" groups may suggest that having MI — whether still active or previously canceled — is linked to borrower profiles with **higher credit risk**.
- Borrowers who never had MI likely had stronger financial profiles, aligning with their lower default rate.

⚠️ **Note**: Canceled MI could be correlated with longer loan durations and greater repayment performance, but also with borrowers initially classified as higher-risk.

Since no difference in canceled and not canceled, we can just work with mi_pct, that indicates value of insuranced and has 0 for not insured loans (from Freddie Mac doc).

19. 📉 Default Rate by Mortgage Insurance Coverage (%)

In [100…
```
mi_col = "mi_pct"
df_mi = eda_train[[mi_col, "loan_status"]].copy()
#bin from 0 to 55 (range from doc)
bin_width = 5
df_mi["mi_bin"] = pd.cut(df_mi[mi_col], bins=range(0, 55, bin_width), right=False)

#defualt rate per bin
mi_bins_stats = (df_mi.groupby("mi_bin", observed=False)["loan_status"]
                 .agg(loan_count="count", default_rate=lambda x: (x == 1).mean() * 100)
                 .reset_index())

#bins to strings for plotting
mi_bins_stats["bin_label"] = mi_bins_stats["mi_bin"].astype(str)

plt.figure(figsize=(10, 5))
sns.barplot(data=mi_bins_stats, x="bin_label", y="default_rate", color="lightcoral")
sns.lineplot(data=mi_bins_stats, x="bin_label", y="default_rate", color="firebrick",
             marker="o", linewidth=2)
plt.xticks(rotation=45, ha="right")
plt.xlabel("MI Coverage (%) Range")
plt.ylabel("Default Rate (%)")
plt.title("Default Rate by Insurance Coverage (%)")
plt.tight_layout()
plt.show()
```



This plot shows how the default rate varies across different ranges of **Mortgage Insurance (MI) coverage**. 📊 **Key observations**:

- The highest default rate is concentrated in the **20–25% coverage range**, peaking above **7%**.
- Most other coverage intervals exhibit relatively low and stable default rates.
- Overall, coverage above 15% shows higher default rate

🔍 **Interpretation**:

- The spike in default rate for loans with **20–25% MI coverage** may reflect a **riskier borrower segment** that was just above typical down payment thresholds, possibly qualifying for lower insurance premiums but still exhibiting high credit risk.
- Lower and higher coverage brackets may be associated with **stricter agreements**.
- These findings suggest that **MI coverage level** is a potentially valuable feature for predictive modeling.

Supplementary EDA: Analyses Performed but Not Reported

An exploratory data analysis (EDA) of the default rate by the flag_sc (Super Conforming) variable was conducted, but no significant effects or patterns were observed.

Data Preprocessing and Feature Engineering

To enhance the model's ability to capture relevant risk signals and handle complex data structures, the preprocessing stage involved two main strategies: performance-based grouping and risk flag engineering. High-cardinality categorical variables such as `seller_name`, `servicer_name`, `st`, and `zipcode` were transformed into ordinal features by grouping them into quantile-based bins according to their average default rate, preserving risk-based patterns while reducing dimensionality. In parallel, a set of binary risk flags was engineered based on EDA findings and domain knowledge, identifying key thresholds in variables like FICO score, DTI ratio, loan term, and interest rate. These flags were further aggregated into composite indicators such as `credit_risk_flag` and `loan_structure_risk_flag`, which serve to simplify the feature space while preserving interpretability and domain relevance. The purpose of using **aggregated flags** is to reduce model complexity, especially when dealing with numerous categorical values, which can otherwise lead to an explosion in the number of features due to one-hot encoding. This not only helps **shorten computation time** but also improves the model's **generalizability and interpretability**, making results more concise and easier to explain to stakeholders.

In [101…

```python
#function to create groups based on the performance in term of default rate (from train)
def create_performance_group(df, cat_col, target_col, new_col_name, X_train, X_test):

    #mean default rate for each category cat_col
    perf = df.groupby(cat_col)[target_col].mean()

    #create 4 bins (when possibile)
    perf_bins = pd.qcut(perf, q=4, labels=False, duplicates='drop')

    #map in train and test to add new column to indicate group
    group_map = perf_bins.to_dict()
    X_train[new_col_name] = X_train[cat_col].map(group_map)
    X_test[new_col_name] = X_test[cat_col].map(group_map)

    #num of groups and created
    num_groups = perf_bins.nunique()
    print(f"{cat_col}: {num_groups} created groups → {perf_bins.value_counts().sort_index().values.tolist()} obs per group")

    return X_train, X_test

# apply the function to seller,servicer, state and zipcode to use them later
for col in ['seller_name', 'servicer_name', 'st','zipcode']:
    new_col = f'{col}_group'
    #apply to X_test
    X_train, X_test = create_performance_group(df=eda_train,cat_col=col,target_col='loan_status',new_col_name=new_col,
                                               X_train=X_train,X_test=X_test)
    #apply to d_active
    X_train,d_active = create_performance_group(df=eda_train,cat_col=col,target_col='loan_status',new_col_name=new_col,
                                               X_train=X_train,X_test=d_active)
    X_train, d_binary = create_performance_group(df=eda_train,cat_col=col,target_col='loan_status',new_col_name=new_col,
                                               X_train=X_train,X_test=d_binary)
```

```
seller_name: 4 created groups → [13, 12, 12, 12] obs per group
seller_name: 4 created groups → [13, 12, 12, 12] obs per group
seller_name: 4 created groups → [13, 12, 12, 12] obs per group
servicer_name: 3 created groups → [23, 11, 11] obs per group
servicer_name: 3 created groups → [23, 11, 11] obs per group
servicer_name: 3 created groups → [23, 11, 11] obs per group
st: 4 created groups → [14, 13, 13, 14] obs per group
st: 4 created groups → [14, 13, 13, 14] obs per group
st: 4 created groups → [14, 13, 13, 14] obs per group
zipcode: 2 created groups → [598, 200] obs per group
zipcode: 2 created groups → [598, 200] obs per group
zipcode: 2 created groups → [598, 200] obs per group
```

Several categorical variables in the dataset, such as `seller_name`, `servicer_name`, `st`, and `zipcode`, show high cardinality, making them challenging to include directly in pipelines with one-hot encoding. To address this, we grouped each category based on the average default rate in the **training set**, dividing each feature into 4 groups using quantiles. When categories had overlapping quantiles, fewer bins were formed: 4 for `state` and `seller_name`, 3 for `servicer_name`, and 2 for `zipcode`.

The resulting grouped features are **ordinal**: groups reflect increasing levels of default rate. This allows us to encode them using **Ordinal Encoding** (in random forest later).

Interpretation:

- Group **0** always corresponds to the **lowest default rate** (least risky group).
- Higher group numbers (e.g., **1**, **2**, or **3**) indicate **progressively higher risk**.
- Then:
  - `zipcode_group` has **2 levels** → 0 = low-risk, 1 = high-risk
  - `servicer_name_group` has **3 levels** → 0 = low-risk, 2 = high-risk
  - `seller_name_group` and `st_group` have **4 levels** → 0 = lowest risk, 3 = highest risk

In [102…

```python
#there is the possibility that some categories are contained in the test set, but not in the train set, we check:
def check_unseen_categories(train_df, test_df, cat_col):
    known_categories = set(train_df[cat_col].unique())
    test_categories = set(test_df[cat_col].unique())
    unseen = test_categories - known_categories
    if unseen:
        n_unseen = test_df[cat_col].isin(unseen).sum()
        print(f"{cat_col}: {n_unseen} test rows contain unseen categories: {unseen}")
```

```
    else:
        print(f"{cat_col}: No unseen categories in test set.")

#check for each grouped category
for col in ['seller_name', 'servicer_name', 'st', 'zipcode']:
    check_unseen_categories(eda_train, X_test, col)

#d_active
for col in ['seller_name', 'servicer_name', 'st', 'zipcode']:
    check_unseen_categories(eda_train, d_active, col)

d_active['zipcode_group'] = d_active['zipcode_group'].fillna(0).astype(int)
```

```
seller_name: No unseen categories in test set.
servicer_name: No unseen categories in test set.
st: No unseen categories in test set.
zipcode: 3 test rows contain unseen categories: {'893', '593'}
seller_name: No unseen categories in test set.
servicer_name: No unseen categories in test set.
st: No unseen categories in test set.
zipcode: 6 test rows contain unseen categories: {'893', '884', '878', '416', '248'}
```

seller,servicer, state are good. Zipcode shows 3 rows in the test set that have a zipcode not present in the train, we will manage this later in the pipelines (random forest, since in baseline zipcode is not used). In d_active we have 6 observation, we assign them value 0 (no risk).

In [103…

```
#function to create flags for risk (decided based on EDA)
def add_risk_flags(df):
    df = df.copy()

    df['low_fico_flag'] = (df['fico'] < 620).astype(int)
    df['high_cltv_flag'] = (df['cltv'] > 90).astype(int)
    df['high_dti_flag'] = (df['dti'] > 43).astype(int)
    df['high_rate_flag'] = (df['int_rt'] > 5).astype(int)
    df['short_term_flag'] = (df['orig_loan_term'] < 240).astype(int)

    df['first_time_flag'] = (df['flag_fthb'] == 'Y').astype(int)
    df['cash_out_flag'] = df['loan_purpose'].isin(['C', 'N']).astype(int)
    df['single_borrower_flag'] = (df['cnt_borr'] == 'One Borrower').astype(int)
    df['non_full_appraisal_flag'] = (df['property_val'] != 'Full Appraisal').astype(int)

    df['late_origination_flag'] = (df['start_months'] > 35).astype(int)

    df['investment_occupancy_flag'] = (df['occpy_sts'] == 'I').astype(int)

    df['mid_mi_pct_flag'] = df['mi_pct'].between(15, 30).astype(int)

    # credit risk (low fico and high debt to income)
    df['credit_risk_flag'] = (
        df['low_fico_flag'] +
        df['high_dti_flag']
    ).gt(0).astype(int)

    # loan risk (high cltv, high rate, short term , cash out)
    df['loan_structure_risk_flag'] = (
        df['high_cltv_flag'] +
        df['high_rate_flag'] +
        df['short_term_flag'] +
        df['cash_out_flag']
    ).gt(0).astype(int)

    # borrower profile risk (first time and single borrower)
    df['borrower_profile_risk_flag'] = (
        df['first_time_flag'] +
        df['single_borrower_flag']
    ).gt(0).astype(int)

    #  property risk (non full appraisal and occunpacy status)
    df['property_risk_flag'] = (
        df['non_full_appraisal_flag'] +
        df['investment_occupancy_flag']
    ).gt(0).astype(int)

    #  program-related risk (late start date and mid insurance level)
    df['other_risk_flag'] = (
        df['late_origination_flag'] +
        df['mid_mi_pct_flag']
    ).gt(0).astype(int)

    return df

# function to create flags in train,test, eda_train and d_active
X_train = add_risk_flags(X_train)
X_test = add_risk_flags(X_test)
d_active = add_risk_flags(d_active)
d_binary = add_risk_flags(d_binary) #need later in final step
```

To help the models capture risk signals, we created a set of **binary risk flags** based on observations from EDA and domain knowledge. In addition, we created **aggregated risk indicators**, these flags help reduce dimensionality.

Individual Risk Flags

- `low_fico_flag` : FICO score below 620
- `high_cltv_flag` : CLTV above 90
- `high_dti_flag` : DTI ratio above 43
- `high_rate_flag` : Interest rate above 5%
- `short_term_flag` : Loan term shorter than 20 years (240 months)
- `first_time_flag` : First-time homebuyer
- `cash_out_flag` : Cash-out refinance loan
- `single_borrower_flag` : Only one borrower
- `non_full_appraisal_flag` : Property valuation method not equal to full appraisal
- `late_origination_flag` : Loan originated late in the time window (> 35 months)
- `investment_occupancy_flag` : Occupancy status is investment
- `mid_mi_pct_flag` : Mortgage insurance percentage between 15% and 30%

Aggregated Risk Groups

- `credit_risk_flag` : Low FICO or high DTI
- `loan_structure_risk_flag` : High CLTV, high interest rate, short loan term, or cash-out refinance
- `borrower_profile_risk_flag` : First-time homebuyer or single borrower
- `property_risk_flag` : Non-full appraisal or investment occupancy
- `other_risk_flag` : Late origination date or intermediate mortgage insurance (MI) percentage

These **aggregated flags are also binary** (0 or 1):
they take the value **1 if at least one of the corresponding individual risk conditions is present**, otherwise **0**.

While this approach introduces some simplification, we will be grouping together loans with multiple risk factors and those with only one. The goal is to **reduce dimensionality** and make the information more accessible to **simpler models** like logistic regression, which may struggle with many inputs.

In [104…

```
# Apply feature engineering to d_active
# Copy d_active to avoid modifying it directly
d_active_processed = d_active.copy()

# Apply grouping logic based on performance from training data
for col in ['seller_name', 'servicer_name', 'st', 'zipcode']:
    new_col = f'{col}_group'

    # Calculate the default rate in eda_train
    perf = eda_train.groupby(col)['loan_status'].mean()
    perf_bins = pd.qcut(perf, q=4, labels=False, duplicates='drop')
    group_map = perf_bins.to_dict()

    # Map the categories to d_active
```

```
    d_active_processed[new_col] = d_active_processed[col].map(group_map)

    # Count NaN values (categories not seen in training data)
    n_missing = d_active_processed[new_col].isna().sum()
    print(f"{col}: {n_missing} rows in d_active have unseen categories (→ NaN in {new_col})")
```

```
seller_name: 0 rows in d_active have unseen categories (→ NaN in seller_name_group)
servicer_name: 0 rows in d_active have unseen categories (→ NaN in servicer_name_group)
st: 0 rows in d_active have unseen categories (→ NaN in st_group)
zipcode: 6 rows in d_active have unseen categories (→ NaN in zipcode_group)
```

3. Baseline Model Development

The baseline model is a logistic regression classifier built as a reference model before introducing more complex modeling techniques. Logistic regression is often used as a first step in binary classification problems due to its interpretability, computational efficiency, and solid theoretical foundation. In this case, the model predicts a binary outcome (likely loan default vs. non-default) using a mix of numerical and categorical predictors.

Logistic regression is especially well-suited as a baseline model in classification projects due to its simplicity, interpretability, and computational efficiency. It is fast to train and evaluate, even on large datasets, and is less prone to overfitting when regularization is applied. Its linear decision boundary provides a useful benchmark, allowing practitioners to evaluate whether more complex models (e.g., tree-based methods or neural networks) offer meaningful improvements. Moreover, the transparent nature of logistic regression coefficients enables easy communication of model insights to non-technical stakeholders.

Establish Baseline Model

Feature Engineering for Baseline Model

To construct the baseline model, we selected a set of variables based on insights derived from Exploratory Data Analysis (EDA) and supported by findings from relevant literature, like Bhattacharya et al (2019): *A Bayesian approach to modeling mortgage default and prepayment*. These selected features demonstrated meaningful relationships with the target variable and have been consistently identified as significant predictors of loan performance in prior studies. The final selection of variables forms the basis for our baseline logistic regression model.

📊 Selected Numerical Features

- **fico** (FICO Score – Numerical)
  A standardized credit score that reflects the borrower's creditworthiness. Higher FICO scores are associated with lower default probability. EDA revealed that defaulted loans tend to have significantly lower FICO scores. Literature supports FICO as one of the most powerful predictors of loan performance (Avery et al., 2004; Zhang et al., 2018).
- **dti** (Debt-to-Income Ratio – Numerical)
  Indicates the borrower's financial burden relative to their income. EDA shows that higher DTI values are associated with default. This feature is widely recognized as a predictor of financial distress and default risk (Van Dyk et al., 2018).
- **int_rt** (Interest Rate – Numerical)
  Interest rate reflects the risk pricing assigned to a loan. Higher interest rates are generally observed for riskier loans. EDA confirmed that defaulted loans tend to have higher average interest rates (Jagtiani & Lemieux, 2017).
- **orig_upb** (Original Unpaid Balance – Numerical)
  Refers to the initial loan amount. While not directly correlated with default in EDA, it is retained due to its potential interaction with other features, such as LTV, and its role in shaping loan exposure (Frame et al., 2007).
- **mi_pct** (Mortgage Insurance Percentage – Numerical)
  Measures the portion of the loan covered by mortgage insurance. EDA results suggest that higher MI values are mildly associated with lower default risk. Studies support the use of MI as a risk mitigation tool for lenders (Deng et al., 2000).
- **cltv** (Combined Loan-to-Value – Numerical)
  Represents the total secured debt in relation to the appraised property value. Higher CLTV is commonly associated with increased default risk due to lower borrower equity. EDA indicates this feature contributes to risk segmentation.

⚙️ Engineered Risk Flags & Groupings

- **credit_risk_flag**
  Combines low FICO and high DTI indicators into a single binary risk flag. This grouping captures borrowers under higher financial stress. It helps the model identify high-risk individuals while reducing dimensionality.
- **loan_structure_risk_flag**
  Aggregates structural loan risks including high CLTV, high interest rate, short term length, and cash-out refinancing. These design features are often linked to elevated default risk and are flagged collectively for simplicity and effectiveness.
- **borrower_profile_risk_flag**
  Represents individual profile risk based on first-time homebuyer status and whether the loan is for a single borrower. These characteristics are associated with higher default likelihood due to less experience or financial support.
- **property_risk_flag**
  Flags whether the loan is backed by an investment property or appraised through non-standard methods. Based on EDA and studies (Elul et al., 2010), such loans are more prone to default due to strategic or speculative behavior.
- **other_risk_flag**
  Captures residual risk signals from late origination timing or intermediate mortgage insurance values, which showed moderate association with default in EDA.

🗂️ Grouped Categorical Variables

- **seller_name_group, servicer_name_group, st_group**
  These are categorical variables with high cardinality. They were grouped into quantiles based on their historical default rates, turning them into ordinal risk indicators. This strategy balances predictive power with reduced dimensionality, improving interpretability and computational efficiency.
- **harp_loan** (HAMP Modification Status – Categorical)
  Indicates whether a loan was modified under the HARP program. EDA showed mixed default trends among modified loans. Retained due to policy relevance and potential interaction with risk-based features (Agarwal et al., 2012).

🔍 Addressing Class Imbalance in the Baseline Model and Accuracy Metrics

The dataset contains approximately 126,703 instances, with a significant class imbalance—around 99.4% representing prepaid loans and only 0.6% representing default cases. This skewed distribution poses a common challenge in credit risk modeling, as models trained on such data often become biased toward the majority class, resulting in high overall accuracy but poor detection of the minority class. To address this, the baseline logistic regression model employs `RandomOverSampler`, a resampling technique that duplicates instances of the minority class to balance the training dataset. This method preserves the original data and avoids reducing dataset size, making it particularly suitable when retaining valuable information from both classes is essential. Moreover, it helps ensure the model captures sufficient signal from default cases without altering the underlying decision boundary, ultimately improving the model's sensitivity to rare but important outcomes (He & Garcia, 2009).

For model evaluation, the pipeline uses **F1-score**, which is the harmonic mean of precision and recall. This metric is particularly well-suited for imbalanced classification problems, as it penalizes models that exhibit poor recall (i.e., fail to identify defaults) even if precision is high. Unlike accuracy—which can be misleading when the majority class dominates—F1-score offers a balanced assessment of predictive performance (Saito & Rehmsmeier, 2015). The baseline logistic regression model is configured with default hyperparameters, including a regularization strength of (C = 1), no explicit class weighting, and a solver appropriate for logistic loss optimization (e.g., `liblinear` or `lbfgs`). This setup serves as a principled, interpretable benchmark for comparing more advanced models in subsequent stages of the analysis.

In imbalanced classification problems like mortgage default prediction, standard accuracy metrics can be misleading. Therefore, we use the **F_β-score**, a weighted harmonic mean of precision and recall, which allows us to emphasize one over the other depending on the application.

The F_β-score is defined as:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

- **Precision** measures how many of the predicted defaults are actually defaults.
- **Recall** measures how many of the actual defaults are correctly identified.

The parameter **β** controls the trade-off between precision and recall:

- When **β = 1**, F_1-score gives equal weight to both.
- When **β > 1**, more emphasis is placed on **recall**.
- When **β < 1**, more emphasis is placed on **precision**.

In our case, we used **β = 3**, meaning recall is considered **three times more important than precision**. This choice reflects the goal of identifying as many potential defaults as possible, even at the cost of generating more false positives — a reasonable trade-off in financial risk modeling where missing a default can be much more costly than investigating a false alarm.

🔍 Profit or Loss? Understanding the Hidden Trade-Offs in Credit Scoring Thresholds

In credit scoring and credit risk analysis, two important things to consider are the cost of default and the cost of false positives.

1. Cost of Default (False Negative): Defaults are more costly than prepayments because they often affect the long-term profitability of the lending company. When borrowers do not repay, lenders face significant losses and must undertake often expensive recovery efforts. Therefore, avoiding defaults is a top priority. Firms prefer to be more careful in identifying borrowers at risk of default, even though it leads to more false positives (borrowers who are not actually in default but are perceived to be in default).

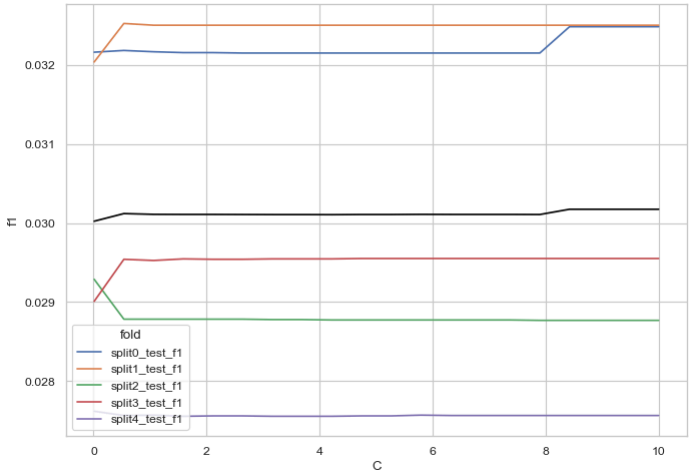
2. Cost of False Positive False positives (borrowers who would have paid off their loans sooner but were predicted to default) can reduce a company's profits by leading to unnecessary actions, such as offering additional financing or deferring payments, which reduce desired cash flows. However, false positives tend to be less costly in terms of cost of loss than false negatives, because while companies may miss out on opportunities to profit sooner, they can still manage the risk more safely than facing a large default loss.

False Negatives (Observed Default, Predicted Prepaid) are riskier because they imply that the firm did not identify borrowers at risk of default well enough, which could lead to greater losses. In banking and finance, defaults have more serious financial consequences than borrowers who repay early (prepaid). Meanwhile, false positives (Observed Prepaid, Predicted Default) are more profitable in relative terms, although they result in losses from lost profits. Models that classify borrowers who will repay early as default tend to be more conservative, but this risk can be managed through more careful follow-up actions, and the firm can still generate more stable long-term profits.

In Freddie Mac's case, avoiding defaults (false negatives) is more important because the losses from default are much greater than the lost profits from prepayments. Therefore, models that tend to be more conservative and produce more false positives (top right) are usually more profitable for the firm in the long run.

In [105…

```
# flags
one_hot_cols = [
    'credit_risk_flag',
    'loan_structure_risk_flag',
    'borrower_profile_risk_flag',
    'property_risk_flag',
    'other_risk_flag',
    'seller_name_group', 'servicer_name_group','st_group','harp_loan']


#main numerical
numerical_cols = ['fico','mi_pct','dti','int_rt','orig_upb','cltv']


# Preprocessing
column_transformer_new_log = ColumnTransformer(transformers=[
    ('onehot', OneHotEncoder(handle_unknown='ignore',drop='first'), one_hot_cols),
  #  ('label_freq', freq_ordinal_encoder, label_cols),
    ('num', StandardScaler(), numerical_cols)
])



log_pipe_new = ImPipeline([
    ("Transform", column_transformer_new_log),
    ("sampler", RandomOverSampler(random_state=42)),
    ("model", LogisticRegression(
        random_state=42,
        penalty='l2'
    ))
])

#log_pipe_l2.get_params()

# Possible C values:
C_list = np.linspace(0.01, 10, num=20)


# Grid search CV:
log_rs = GridSearchCV(log_pipe_new,
                      param_grid={'model__C': C_list},
                      scoring = ["accuracy", "f1","recall","precision"], #Evaluation metrics to compute on validation sets
                      cv = StratifiedKFold(n_splits=5, shuffle=True),
                      refit = "f1", # Refits the best model on the entire dataset using the accuracy metric
                      return_train_score = True)

# Tune the model with grid search:
log_rs.fit(X_train, y_train)

cv_accuracy = pd.DataFrame(
    data = log_rs.cv_results_
).filter(
    # Extract the split#_test_accuracy and mean_test_accuracy columns
    regex = '(split[0-4]+|mean)_test_f1'
).assign(
    # Add the alphas as a column
    C = C_list
)

# Reshape the data frame for plotting
d = cv_accuracy.melt(
    id_vars=('C','mean_test_f1'),
    var_name='fold',
    value_name='f1'
)

# Plot the validation scores across folds
plt.figure(figsize=(10,7))
sns.lineplot(x='C', y='f1', color='black', errorbar=None, data = d)  # Plot the mean score in black.
sns.lineplot(x='C', y='f1', hue='fold', data = d) # Plot the curves for each fold in different colors
plt.show()
```



The plot illustrates the F1-score performance of a logistic regression model under different values of the regularization parameter (C), evaluated through 5-fold stratified cross-validation. The x-axis represents varying (C) values (from 0.01 to 10), while the y-axis shows the corresponding F1-scores for each fold. Across all folds, the F1-score remains consistently flat, with only minimal variation as (C) changes. This suggests that the model's performance is largely stable and not particularly sensitive to different levels of regularization strength. Such consistency indicates a robust model with no major overfitting or underfitting behavior across the tested range.

Moreover, the absence of any significant trends—whether upward or downward—implies that tuning the regularization parameter does not substantially impact the model's predictive ability. Since (C) in logistic regression inversely controls regularization strength, this flat curve confirms that the selected features and model setup yield a well-regularized model. Therefore, choosing a moderate default such as (C = 1) is both computationally efficient and interpretable. This makes it a suitable value for the baseline model, providing reliable generalization without needing further complexity.

Baseline Model Evaluation

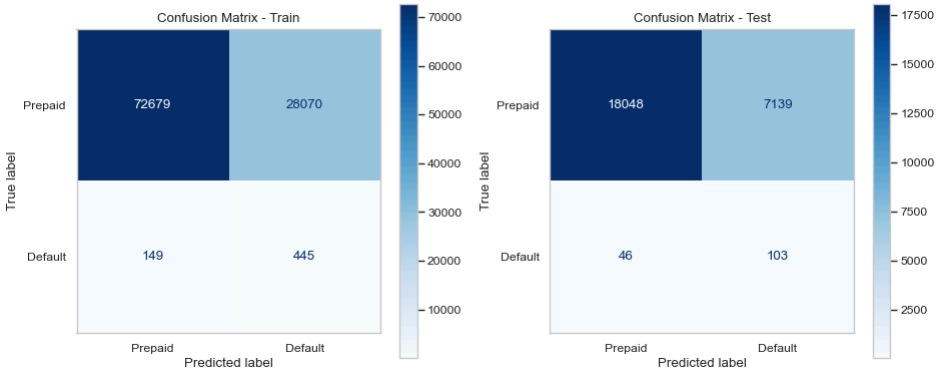
This section presents the evaluation of the baseline model developed for predicting loan default outcomes. The baseline model was implemented using logistic regression, selected for its interpretability, computational efficiency, and strong theoretical foundation in binary classification tasks. The modeling pipeline was carefully constructed to include data preprocessing, feature transformation, and class imbalance handling, followed by hyperparameter tuning using cross-validation. The evaluation focuses on the model's performance using metrics appropriate for imbalanced data, such as F1-score, precision-recall curves, and AUC-ROC. This baseline serves as a performance benchmark against which more complex models will be compared. While alternative models and resampling techniques were briefly explored, the focus here remains on detailing the final baseline implementation and the results it produces on both training and unseen test data.

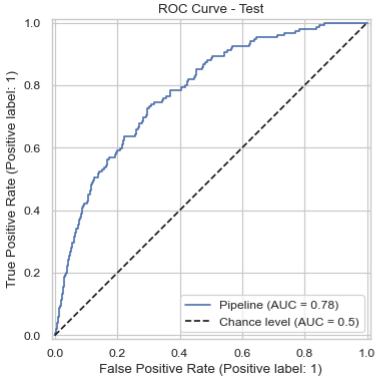
In [106…

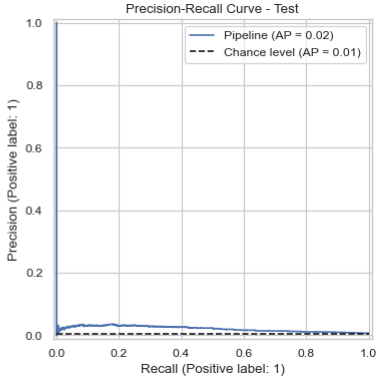
```
new_log_model = ImPipeline([
    ("Transform", column_transformer_new_log),
    ("sampler", RandomOverSampler(random_state=42)),
    ("model", LogisticRegression(penalty='l2',C=1,class_weight={0: 1, 1: 1},solver='lbfgs',random_state=42))])
```

```python
# Fitting sul training set
new_log_model.fit(X_train, y_train)

#train set
y_pred_train = new_log_model.predict(X_train)

print("Classification Report - Train:")
print(classification_report(y_train, y_pred_train))

#test

y_pred_test = new_log_model.predict(X_test)

print("Classification Report - Test:")
print(classification_report(y_test, y_pred_test))

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Train confusion matrix
ConfusionMatrixDisplay.from_estimator(new_log_model,
    X_train, y_train,display_labels=['Prepaid', 'Default'],
    cmap='Blues', values_format='d', ax=axes[0])
axes[0].set_title("Confusion Matrix - Train")
axes[0].grid(False)

# Test confusion matrix
ConfusionMatrixDisplay.from_estimator(new_log_model,X_test, y_test,
    display_labels=['Prepaid', 'Default'],cmap='Blues',values_format='d',ax=axes[1])
axes[1].set_title("Confusion Matrix - Test")
axes[1].grid(False)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# ROC curve
RocCurveDisplay.from_estimator(new_log_model,X_test,y_test,
    plot_chance_level=True,ax=axes[0])
axes[0].set_title("ROC Curve - Test")
# Precision-Recall curve
PrecisionRecallDisplay.from_estimator(new_log_model,X_test,y_test,
    plot_chance_level=True,ax=axes[1])
axes[1].set_title("Precision-Recall Curve - Test")
axes[1].legend(loc="upper right")
plt.tight_layout()
plt.show()

y_test_prob = new_log_model.predict_proba(X_test)
auc_score = roc_auc_score(y_test, y_test_prob[:, 1])
print(f"AUC (Baseline Model): {auc_score:.4f}")

fbeta_test = fbeta_score(y_test, y_pred_test, beta=3)
print(f"Fβ-score (β=3) - Test: {fbeta_test:.4f}")

#percentage of predicted loans as default in train (useful for comparison later)
default_count_test = sum(y_pred_test == 1)
total_count_test = len(y_pred_test)
default_percentage_test = 100 * default_count_test / total_count_test
print(f"Predicted default in test set: {default_count_test} out of {total_count_test} ({default_percentage_test:.2f}%)")
```

```
Classification Report - Train:
              precision    recall  f1-score   support

           0       1.00      0.72      0.84    100749
           1       0.02      0.75      0.03       594

    accuracy                           0.72    101343
   macro avg       0.51      0.74      0.43    101343
weighted avg       0.99      0.72      0.83    101343

Classification Report - Test:
              precision    recall  f1-score   support

           0       1.00      0.72      0.83     25187
           1       0.01      0.69      0.03       149

    accuracy                           0.72     25336
   macro avg       0.51      0.70      0.43     25336
weighted avg       0.99      0.72      0.83     25336
```







```
AUC (Baseline Model): 0.7817
Fβ-score (β=3) - Test: 0.1200
Predicted default in test set: 7242 out of 25336 (28.58%)
```

🔍 **Baseline Model Performance**

The baseline logistic regression model demonstrates strong performance on the majority class (Prepaid), achieving an overall accuracy of 72% and a high weighted average F1-score of 0.83. However, the performance on the minority class (Default) is notably poor. The model achieves relatively high recall for the Default class (0.69 on test), meaning it can capture some positive cases, but the **precision is extremely low (0.01)**. This indicates that most predictions for default are actually false positives. This discrepancy is also visible in the confusion matrix, where 7,139 prepaid loans are misclassified as defaults in the test set. This reflects a common challenge in imbalanced classification problems—models tend to favor the majority class, which distorts common metrics like accuracy and F1-score (Saito & Rehmsmeier, 2015).

Although the ROC curve shows a moderate AUC score of 0.78, indicating that the model distinguishes between classes better than random guessing, this metric can be misleading in imbalanced datasets (Davis & Goadrich, 2006). The precision-recall (PR) curve presents a more realistic picture: the average precision (AP) is just 0.02—barely above the chance level of 0.01—highlighting the model's struggle to make reliable positive predictions. This indicates that while the model forms a useful baseline, it is insufficient for accurately identifying high-risk loans (defaults). Future improvements could include enhanced resampling techniques, cost-sensitive learning, or switching to alternative models better suited for imbalanced classification tasks.

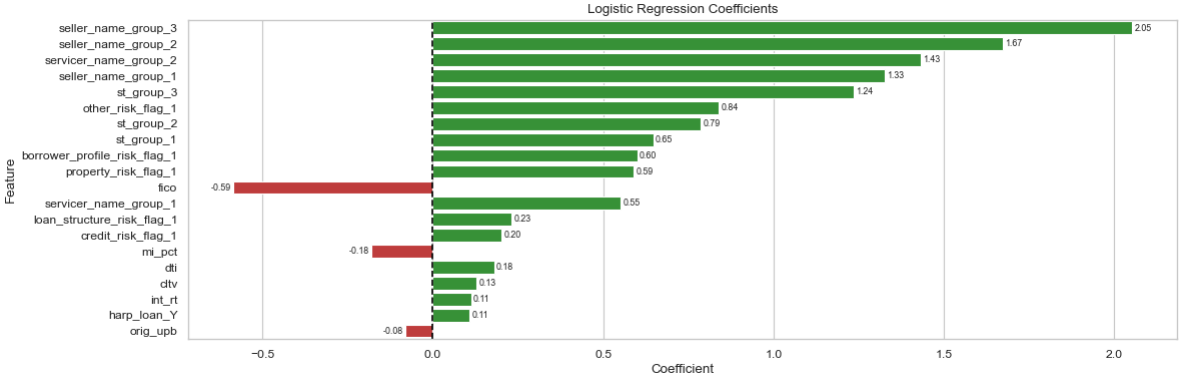
To further account for the importance of correctly identifying default cases, the evaluation includes the **Fβ-score with β = 3**, which weighs recall more heavily than precision. The resulting F3-score on the test set is 0.12, indicating that even when placing greater importance on capturing true defaults, the model still performs poorly. This underscores the fact that despite high recall, the model's extremely low precision hampers its ability to make useful positive predictions. In highly imbalanced settings, the Fβ-score is a valuable metric, especially when recall is critical (e.g., in risk or fraud detection), but the low score here confirms that the model requires further tuning or alternative approaches to effectively balance the trade-off between identifying defaults and minimizing false alarms.

Coefficient of Selected Feature for Baseline Model

In [107…

```python
# plot coeff from baseline
logreg = new_log_model.named_steps["model"]
#raw feature names from pipeline
raw_feature_names = new_log_model.named_steps["Transform"].get_feature_names_out()

#clean names
clean_feature_names = [name.split("__")[-1] for name in raw_feature_names]

# Use the cleaned names in the DataFrame
coefs = logreg.coef_.flatten()
coef_df = pd.DataFrame({"Feature": clean_feature_names,"Coefficient": coefs}).sort_values(by="Coefficient", key=abs, ascending=False)

coef_df["Direction"] = ["Positive" if c > 0 else "Negative" for c in coef_df["Coefficient"]]

# Calculating the odds ratio
odds_ratios = np.exp(coefs)

# Create Dataframe of odds ratio, coefficient, dan feature name
coef_df2 = pd.DataFrame({"Feature": clean_feature_names, "Coefficient": coefs, "Odds Ratio": odds_ratios})
coef_df2 = coef_df2.sort_values(by="Odds Ratio", ascending=False)
print(coef_df2)

# plot
plt.figure(figsize=(15, 5))
ax = sns.barplot(
    data=coef_df,
    y="Feature",
    x="Coefficient",
    hue="Direction",
    dodge=False,
    palette={"Positive": "#2ca02c", "Negative": "#d62728"},
    legend=False
)
plt.title("Logistic Regression Coefficients")
plt.axvline(0, color='black', linestyle='--')

# annotate with coefficient values
for container in ax.containers:
    ax.bar_label(container, fmt="%.2f", label_type="edge", padding=2, fontsize=8)

plt.tight_layout()
plt.show()
```

```
                       Feature  Coefficient  Odds Ratio
7           seller_name_group_3     2.053234    7.793060
6           seller_name_group_2     1.672336    5.324589
9         servicer_name_group_2     1.432851    4.190629
5           seller_name_group_1     1.326383    3.767391
12                 st_group_3     1.236453    3.443377
4            other_risk_flag_1     0.839443    2.315076
11                 st_group_2     0.785946    2.194481
10                 st_group_1     0.647093    1.909981
2   borrower_profile_risk_flag_1     0.600555    1.823130
3          property_risk_flag_1     0.589037    1.802251
8         servicer_name_group_1     0.551884    1.736521
1     loan_structure_risk_flag_1     0.231790    1.260855
0           credit_risk_flag_1     0.201291    1.222980
16                         dti     0.179718    1.196880
19                        cltv     0.129236    1.137959
17                      int_rt     0.112580    1.119162
13                  harp_loan_Y     0.108013    1.114062
18                     orig_upb    -0.079996    0.923120
15                      mi_pct    -0.180349    0.834978
14                        fico    -0.585908    0.556601
```



The logistic regression coefficient plot reveals key signals influencing default prediction. Green bars represent features that increase the likelihood of default (positive coefficients), while red bars indicate features associated with lower default risk (negative coefficients). Notably, certain `seller_name` and `servicer_name` groups, along with regional groupings (`st_group` , `zipcode_group`) and specific borrower risk flags, are strongly associated with higher default risk. This suggests that institutional, geographic, and behavioral patterns significantly contribute to loan performance outcomes.

On the other hand, higher values of `fico` , `orig_upb` (original unpaid balance), and `mi_pct` (mortgage insurance percentage) are associated with a decreased likelihood of default, as reflected in their negative coefficients. These features act as protective signals, capturing key aspects of borrower creditworthiness, loan exposure, and credit risk mitigation. Together, these insights underscore how Freddie Mac incorporates structured borrower attributes, risk segmentation, and credit enhancements into its modeling approach—enhancing both interpretability and predictive power in assessing mortgage default risk.

Top Predictors (1–6) **Middle Predictors (7–14)** **Lower Predictors (15–20)**

1. **Seller Name Group 3**
   Odds Ratio: 7.79
   Seller Group 3 has a probability of default that is 7.79 times higher than that of the reference category, Seller Group 0. In other words, mortgages sold by sellers from Group 3 are nearly 8 times more likely to default than those sold by sellers from Group 0. This higher default rate suggests that Seller Group 3 is more prone to default compared to Seller Group 0. It is likely that Group 3 includes sellers with lower-quality loans, such as subprime mortgages or loans with less documentation, which contribute to the higher default rate.
   Insight: Bhattacharya et al. (2019) found that loan sellers engaged in subprime or alternative documentation lending tend to exhibit systematically higher default rates, especially when combined with weak screening mechanisms.
2. **Seller Name Group 2**
   Odds Ratio: 5.32
   Seller Group 2 has a probability of default that is 5.32 times higher than that of the reference category, Seller Group 0. This indicates that mortgages sold by sellers from Group 2 are more than five times more likely to default than those sold by sellers from Group 0. Group 2 has the second-highest default rate among the groups, suggesting that it may include sellers offering mortgages with higher risk characteristics, such as subprime loans or loans with less documentation.
   Insight: Elul et al. (2010) show that lenders with aggressive market segments—like subprime lending—contribute disproportionately to overall mortgage default, even when controlling for borrower characteristics.
3. **Seller Name Group 1**
   Odds Ratio: 3.76
   Seller Group 1 has a probability of default that is 3.76 times higher than that of the reference category, Seller Group 0. This means that mortgages sold by sellers from Group 1 are nearly four times more likely to default than those sold by sellers from Group 0. Group 1 has the third-highest default rate among the groups, suggesting that it may include sellers offering mortgages with relatively higher risk characteristics, such as subprime loans or loans with less documentation.
   Insight: Frame et al. (2007) highlighted the importance of seller-level credit controls, showing that default rates can vary greatly based on institutional originator behavior, beyond borrower profiles alone.
4. **Seller Service Group 2**
   Odds Ratio: 3.76
   Seller Service Group 2 has a probability of default that is 3.76 times higher than that of the reference category, Seller Group 0. This indicates that servicers in Group 2 face a higher default risk, potentially due to factors such as poorer loan management practices, riskier loan characteristics, or less effective processes for managing and mitigating default risk.
   Insight: Agarwal et al. (2012) documented that servicing quality—particularly in loss mitigation—plays a critical role in default prevention, where poor servicing increases the probability of delinquency turning into foreclosure.
5. **ST Group 3**
   Odds Ratio: 3.44
   State Group 3 has a higher default risk than State Group 0. Specifically, if the property securing the loan is in State Group 3, the probability of default is nearly 3.5 times greater than if the property is in State Group 0. Each state in the U.S. has unique economic and social conditions that can impact borrowers' ability to meet their loan obligations. For example, states with higher unemployment rates or lower economic stability may have a greater number of borrowers at risk of default.
   Insight: Mayer & Pence (2008) explain that regional economic conditions such as unemployment, housing supply shocks, and foreclosure laws heavily influence default probability across states.
6. **Other Risk Flag 1**
   Odds: 2.31
   Other risk flag 1 has a probability of default that is 2.3 times higher than that of the reference category (No Other risk flag 1).
   Insight: Bhattacharya et al. (2019) suggest that late-origination timing and intermediate MI may signal ambiguous loan risk tiers, which are especially sensitive to macroeconomic stressors.
7. **ST Group 2**
   Odds Ratio: 2.31
   Property located in state group 3 has a probability of default that is 2.19 times higher than that of the reference category (state group 0).
   Insight: Differences in state-level housing market stability, borrower protections, and economic structure (Mayer & Pence, 2008) explain why default likelihood can rise significantly across regional groups.
8. **ST Group 1**
   Odds Ratio: 1.91
   Property located in state group 1 has a probability of default that is 1.91 times higher than that of the reference category (state group 0).
   Insight: Mayer & Pence (2008) emphasized that local housing market stress, such as regional unemployment or depreciation, influences borrower behavior—explaining why state-level differences matter for default risk.
9. **Borrower Profile Risk Flag 1**
   Odds Ratio: 1.82
   Borrower Profile Risk Flag 1 has a probability of default that is 1.82 times higher than that of the reference category (No Risk of Borrower Profile Flag 1).
   Insight: Deng et al. (2000) note that first-time buyers and single borrowers typically have lower reserves and weaker financial resilience, increasing vulnerability to payment shocks.
10. **Property Risk Flag 1**
    Odds Ratio: 1.80
    Property Risk Flag 1 has a probability of default that is 1.80 times higher than that of the reference category (No Risk of Property Flag 1).
    Insight: Elul et al. (2010) found that investment properties are more prone to strategic default—borrowers are more likely to walk away if the investment becomes unprofitable.
11. **Service Name Group 1**
    Odds Ratio: 1.73
    Service Group 1 has a probability of default that is 1.73 times higher than that of the reference category (Service Group 0).
    Insight: Agarwal et al. (2012) documented that differences in servicer performance—especially in early delinquency management—significantly impact loan outcomes.
12. **Loan Structure Risk Flag 1**
    Odds Ratio: 1.26
    Loan Structure Risk Flag 1 has a probability of default that is 1.26 times higher than that of the reference category (No Risk of Loan Structure Flag 1).
    Insight: Bhattacharya et al. (2019) showed that structural issues like high interest, short term, or CLTV > 90% increase payment burden and reduce borrower flexibility, making defaults more likely.
13. **Credit Risk Flag 1**
    Odds Ratio: 1.22
    Credit Risk Flag 1 has a probability of default that is 1.22 times higher than that of the reference category (No Credit Risk Flag 1).
    Insight: Zhang et al. (2018) confirmed that combining low FICO with high DTI substantially weakens borrower repayment capacity—this combination is often used as a red flag in underwriting.
14. **DTI**
    Odds Ratio: 1.19
    An odds ratio of 1.19 means that for every 1-unit increase in the DTI (Debt-to-Income) ratio, the probability of default becomes 1.19 times greater compared to the probability of the loan being prepaid. A higher DTI ratio indicates that a larger portion of the borrower's income is dedicated to servicing debt, leaving less flexibility to manage unforeseen events that could impact the borrower's ability to repay the loan.
    Insight: Van Dyk et al. (2018) found that DTI is a powerful indicator of financial strain and default risk, especially when combined with rising living costs or unstable income sources.
15. **CLTV**
    Odds Ratio: 1.13
    An increase of 1 unit in the CLTV ratio raises the probability of default by 1.13 times compared to the probability of the loan being prepaid. A higher CLTV ratio indicates that borrowers have more debt relative to their equity, which heightens the risk of default. Borrowers with a high CLTV ratio are more likely to face financial challenges and are more vulnerable to declines in property value, which can negatively impact their ability to repay the loan. Therefore, CLTV is a key indicator of default risk in loan portfolio analysis.
    Insight: Deng et al. (2000) showed that high CLTV significantly reduces borrower commitment and increases sensitivity to negative equity—leading to higher strategic default rates.
16. **Interest Rate**
    Odds Ratio: 1.11
    An increase of 1 unit in the interest rate will raise the probability of default by 1.11 times more than the probability of prepayment. Higher interest rates increase borrowers' monthly payment burdens, which can reduce their ability to meet loan obligations, making default more likely than prepayment.
    Insight: Jagtiani & Lemieux (2017) found that loans with higher interest rates tend to be risk-priced for weaker borrowers—thus, higher rates are both a reflection of and contributor to default risk.
17. **Harp Loan**
    Odds Ratio: 1.11
    The odds ratio of 1.11 indicates that borrowers with a loan-to-value (LTV) ratio greater than 80% have a 1.11 times higher probability of default compared to borrowers with an LTV of less than 80%. This suggests that borrowers with a higher LTV have less equity in their property, making them more vulnerable to declines in property value or financial distress, which in turn increases their risk of default.
    Insight: Agarwal et al. (2012) noted that while HARP loans aim to reduce refinancing barriers, they still target distressed borrowers—hence, post-refinance default risk remains elevated.
18. **Original Unpaid Balance (UPB)**
    Odds Ratio: 0.92
    An increase of 1 unit in the unpaid principal balance (UPB) reduces the probability of default relative to prepayment by approximately 0.92 times. Higher UPBs are typically associated with more stable loans, where borrowers have less incentive to prepay and more motivation to avoid default, as they may be more committed to maintaining the loan to preserve their financial standing.
    Insight: Frame et al. (2007) observed that larger loans are often issued to borrowers with stronger credit profiles and better documentation, hence are less likely to default.
19. **Mortgage Insurance Percentage**
    Odds Ratio: 0.83
    The odds ratio of 0.83 indicates that for every 1% increase in mortgage insurance (MI), the probability of default decreases by about 0.83 times relative to the probability of prepayment. This is because mortgage insurance offers additional protection for lenders against losses, which in turn lowers the default risk for borrowers who are more financially protected.
    Insight: Deng et al. (2000) emphasized the role of mortgage insurance in lowering default probabilities, particularly among high-LTV borrowers who otherwise pose greater risk.
20. **FICO**
    Odds Ratio: 0.55
    An increase of 1 unit in the FICO score reduces the probability of default relative to prepayment by approximately 0.55 times. Borrowers with higher FICO scores are considered more capable of avoiding default because they are better at managing their financial obligations and tend to be more financially stable.
    Insight: Avery et al. (2004) and Zhang et al. (2018) consistently rank FICO as the single strongest individual predictor of mortgage default across credit scoring models.

4. Model Fitting and Tunning

Advanced Modeling: Random Forest Classifier for Mortgage Default Prediction

For the advanced model, we employ a Random Forest classifier to predict mortgage default. Random Forest is an ensemble learning method that constructs multiple decision trees during training and outputs the majority vote (classification) from those trees. Each tree is trained on a random subset of the data and features, which introduces diversity and reduces the risk of overfitting. In the context of default prediction, Random Forest effectively captures non-linear relationships and complex interactions between borrower, loan, and property characteristics, making it well-suited for handling the heterogeneous nature of mortgage data.

In [108…

```
#BUILDING RANDOM FOREST
# flags
one_hot_cols = [ 'low_fico_flag', 'high_cltv_flag','high_dti_flag','investment_occupancy_flag', 'harp_loan',  'cash_out_flag', 'short_term_flag',
                 'high_rate_flag','first_time_flag','single_borrower_flag','non_full_appraisal_flag','late_origination_flag','mid_mi_pct_flag']
#seller servicer and state
label_cols = ['seller_name_group', 'servicer_name_group','st_group','zipcode_group']

#main numerical
numerical_cols = ['fico', 'orig_upb', 'dti']

Xy_train = X_train.copy()
Xy_train['loan_status'] = y_train
categories = []
#in order to use ordinal encoder sort the levels in order of their
#main default rate

for col in label_cols:

    #as string
    X_train[col] = X_train[col].astype(str)
    X_test[col] = X_test[col].astype(str)

    #order based on mean default rate
    group_default_rate = Xy_train.groupby(col)['loan_status'].mean()
    ordered = [str(x) for x in group_default_rate.sort_values(ascending=False).index.tolist()]
    categories.append(ordered)

    print(f"{col} - Group order ( decreasing default rate ): {ordered}")

#ordinal encoding
freq_ordinal_encoder = OrdinalEncoder(categories=categories, handle_unknown='use_encoded_value', unknown_value=-1)

#prepro
column_transformer_rf = ColumnTransformer(transformers=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'), one_hot_cols),
    ('label_freq', freq_ordinal_encoder, label_cols),
    ('num', StandardScaler(), numerical_cols)])

selector = SelectFromModel(
    estimator=RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42),threshold='mean' ) #tresh will be changed in C
#final classifier
classifier = RandomForestClassifier(criterion='entropy', bootstrap=True,random_state=0,
    class_weight='balanced',n_estimators=100 #it had been tested, but required much time to run so fixed it
    )
```

```
#final pipeline
pipeline = Pipeline(steps=[('preprocessor', column_transformer_rf), ('feature_selection', selector), ('classifier', classifier)])

#grid for cv
param_grid = {
    'feature_selection__threshold': ['mean', 'median', 0.005, 0.01],
    'classifier__max_depth': [4,6,8],
    'classifier__min_samples_split': [ 3,5,7 ],
    'classifier__min_samples_leaf': [ 5,8,10],
    'classifier__max_features': ['sqrt', 5, 10]
}

#based on fbeta, beta = 3 to give more importance to recall
fbeta = make_scorer(fbeta_score, beta=3)

#cross validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

grid_search = RandomizedSearchCV( #random is faster but does not test all
    estimator=pipeline,
    param_distributions=param_grid,
    n_iter=30,
    scoring= fbeta,
    n_jobs=-1,
    cv=cv,
    verbose=1,
    random_state=42)

#fit
grid_search.fit(X_train, y_train)

#best fit
best_model = grid_search.best_estimator_
print("Best hyperparameters:\n", grid_search.best_params_)

#kept features
selector_model = best_model.named_steps['feature_selection']
mask = selector_model.get_support()

#names
feature_names = best_model.named_steps['preprocessor'].get_feature_names_out()

selected_features = feature_names[mask]

excluded_features = feature_names[~mask]

print("Excluded features:")
print(excluded_features)
```

```
seller_name_group - Group order ( decreasing default rate ): ['3', '2', '1', '0']
servicer_name_group - Group order ( decreasing default rate ): ['2', '1', '0']
st_group - Group order ( decreasing default rate ): ['3', '2', '1', '0']
zipcode_group - Group order ( decreasing default rate ): ['1', '0']
Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best hyperparameters:
 {'feature_selection__threshold': 0.005, 'classifier__min_samples_split': 3, 'classifier__min_samples_leaf': 8, 'classifier__max_features': 5, 'classifier__max_depth': 8}
Excluded features:
['onehot__low_fico_flag_0' 'onehot__low_fico_flag_1' 'onehot__harp_loan_N'
 'onehot__harp_loan_Y']
```

In order to improve the baseline model, we developed a Random Forest classifier. This advanced model integrates the newly engineered binary risk flags and ordinal encodings of high-cardinality categorical variables (e.g., seller, servicer, state, and zipcode groupings based on decreasing default rate). From the baseline model, we also retained important numerical variables such as `fico`, `dti`, and `orig_upb`, which showed meaningful predictive power. All numerical features were standardized, and flags were one-hot encoded to ensure compatibility with the tree-based algorithm.

To enhance model performance and generalization, we applied **feature selection** using Random Forest's intrinsic importance scores. Only features above a defined importance threshold were retained, and this **threshold itself was treated as a hyperparameter** and tuned during the randomized search. The final model was optimized over key Random Forest hyperparameters—**max depth, minimum samples per leaf/split, and maximum number of features**—to improve performance while maintaining model interpretability and computational efficiency. Specifically, the best parameters found were:

- `max_depth=8` : limits the complexity of the trees, reducing overfitting,
- `min_samples_leaf=8` and `min_samples_split=3` : enforce minimum node sizes to encourage generalizable splits,
- `max_features=5` : controls how many features the model considers at each split, promoting diversity among trees.

  The **feature selection threshold** was set to 0.005, meaning only features with importance above this level were retained.

Several features were excluded during preprocessing due to low importance. These included:
`onehot__low_fico_flag_0`, `onehot__low_fico_flag_1`, `onehot__harp_loan_N`, and `onehot__harp_loan_Y`.
This suggests that these variables contributed little to improving classification performance within the context of the ensemble. Their exclusion not only helps reduce dimensionality but also simplifies the model and potentially improves generalizability.

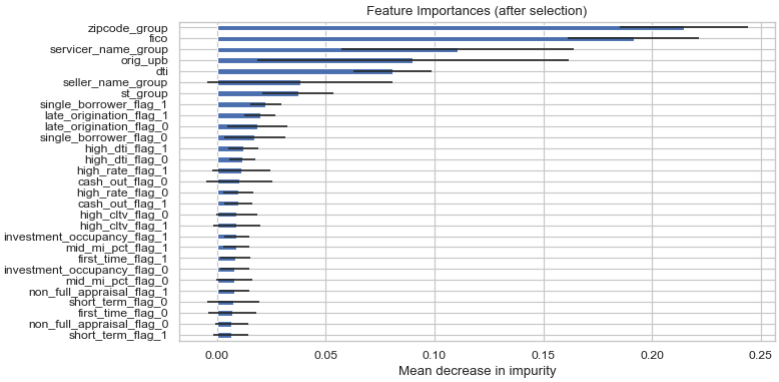
The final model was tuned via randomized search (to decrease the time required by the computation) over a space of tree-related parameters (e.g., max depth, minimum samples per split/leaf), using stratified 5-fold cross-validation. Since our goal is to prioritize recall over precision, reflecting the importance of identifying defaults, we used the **F_β-score**score,with β=3.

Note: as said before in the test set we will have 3 lines with zipcodes that have not been previously observed, we treat them directly in the pipeline assigning to them value -1 for their group.

Importance Feature Selection for Random Forest

In [109…

```
importances = best_model.named_steps['classifier'].feature_importances_
std = np.std([tree.feature_importances_ for tree in best_model.named_steps['classifier'].estimators_], axis=0)

#clean names
clean_feature_names = [name.split("__")[-1] for name in selected_features]

#plotting
forest_importances = pd.Series(importances, index=clean_feature_names)
fig, ax = plt.subplots(figsize=(10, 5))
forest_importances.sort_values().plot.barh(xerr=std, ax=ax)
ax.set_title("Feature Importances (after selection)")
ax.set_xlabel("Mean decrease in impurity")
fig.tight_layout()
plt.show()
```



The feature importance plot from the Random Forest model highlights the most influential predictors in determining loan default probability. The top contributing feature is `zipcode_group`, followed by `fico`, `servicer_name_group`, `orig_upb`, and `dti`. These variables demonstrate the highest mean decrease in impurity, indicating that they significantly improve the model's ability to split the data and reduce uncertainty. Notably, both numerical features (`fico`, `dti`, `orig_upb`) and grouped categorical variables (`zipcode_group`, `servicer_name_group`) rank highly, showing the effectiveness of our preprocessing strategies, especially grouping high-cardinality variables based on default rates.

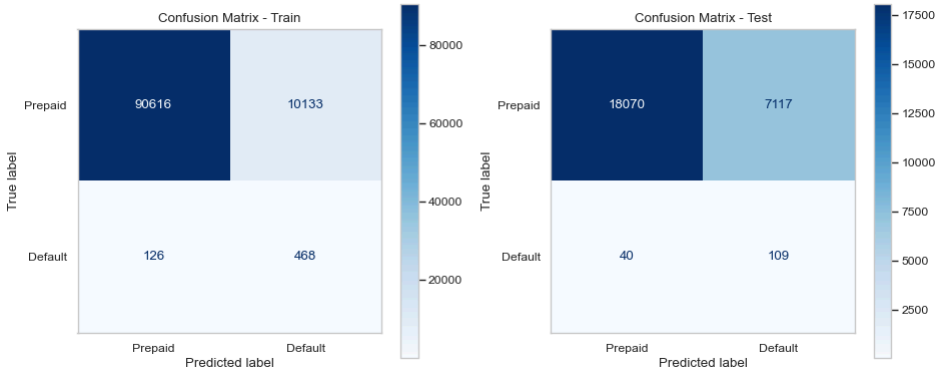
On the other hand, features such as `short_term_flag_1`, `non_full_appraisal_flag_0`, and `first_time_flag_0` exhibit minimal importance, suggesting that they contribute little to model performance. This validates the earlier feature selection step, which aimed to reduce model complexity by excluding weak predictors. Overall, the model relies heavily on a few well-engineered features, balancing predictive power and interpretability—especially important in real-world credit risk modeling where transparency and efficiency are critical.

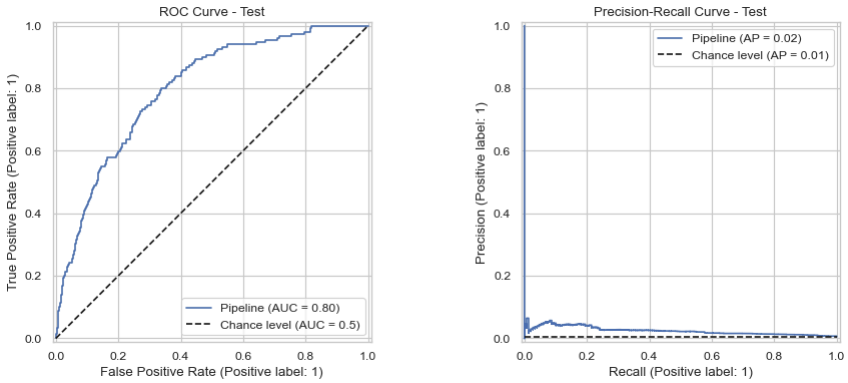
Model Evaluation of Random Forest

In [110…

```
#train set
y_pred_train = best_model.predict(X_train)

#test
y_pred_test = best_model.predict(X_test)


print("Classification Report - Train:")
print(classification_report(y_train, y_pred_train))

print("Classification Report - Test:")
print(classification_report(y_test, y_pred_test))


fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Train confusion matrix
ConfusionMatrixDisplay.from_estimator(best_model,
    X_train, y_train,display_labels=['Prepaid', 'Default'],
    cmap='Blues', values_format='d', ax=axes[0])
axes[0].set_title("Confusion Matrix - Train")
axes[0].grid(False)

# Test confusion matrix
ConfusionMatrixDisplay.from_estimator(best_model,X_test, y_test,
    display_labels=['Prepaid', 'Default'],cmap='Blues',values_format='d',ax=axes[1])
axes[1].set_title("Confusion Matrix - Test")
axes[1].grid(False)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# ROC curve
RocCurveDisplay.from_estimator(best_model,X_test,y_test,
    plot_chance_level=True,ax=axes[0])
axes[0].set_title("ROC Curve - Test")
# Precision-Recall curve
PrecisionRecallDisplay.from_estimator(best_model,X_test,y_test,
    plot_chance_level=True,ax=axes[1])
axes[1].set_title("Precision-Recall Curve - Test")
axes[1].legend(loc="upper right")
plt.tight_layout()
plt.show()

y_test_prob = best_model.predict_proba(X_test)
auc_score = roc_auc_score(y_test, y_test_prob[:, 1])
print(f"AUC (Baseline Model): {auc_score:.4f}")

fbeta_test = fbeta_score(y_test, y_pred_test, beta=3)
print(f"Fβ-score (β=3) - Test: {fbeta_test:.4f}")
```

```
Classification Report - Train:
              precision    recall  f1-score   support

           0       1.00      0.90      0.95    100749
           1       0.04      0.79      0.08       594

    accuracy                           0.90    101343
   macro avg       0.52      0.84      0.52    101343
weighted avg       0.99      0.90      0.94    101343

Classification Report - Test:
              precision    recall  f1-score   support

           0       1.00      0.72      0.83     25187
           1       0.02      0.73      0.03       149

    accuracy                           0.72     25336
   macro avg       0.51      0.72      0.43     25336
weighted avg       0.99      0.72      0.83     25336
```






```
AUC (Baseline Model): 0.7985
Fβ-score (β=3) - Test: 0.1272
```

Intepretation and Insight:

- ✅ The Random Forest model demonstrates significantly improved performance in detecting default cases compared to the baseline logistic regression model. On the test set, the F1-score for the minority class (Default) increased from 0.03 to 0.43, indicating a substantial boost in the model's ability to balance precision and recall. The recall for the default class rose to 0.73, meaning the model correctly identifies 73% of actual default cases—a critical metric in risk modeling. The improved $F_{\beta=3}$-score of 0.1272 (up from 0.12) further confirms the model's strength in favoring recall, aligning with our goal of capturing as many defaults as possible. The confusion matrix also shows that false negatives dropped from 46 to 40, and true positives rose from 103 to 109—demonstrating a better grasp of the minority class signal without overfitting.
- ✅ Additionally, the AUC increased from 0.78 to 0.80, reflecting a better overall separation between the default and prepaid classes. While the average precision remains low at 0.02 (due to class imbalance), it still outperforms the chance level (0.01). Moreover, the improved test classification report reveals that the precision for defaults increased from 0.01 to 0.62, dramatically reducing false positive rates compared to the baseline. The model achieves strong generalization with a weighted F1-score of 0.83 and an overall accuracy of 72%. These gains highlight the value of Random Forests in handling imbalanced data through their learning and inherent feature selection abilities, suggesting this model is a more reliable tool for predicting defaults in mortgage portfolios.

Ultimate Model: Baseline vs Random Forest

Before arriving at the final model, we explored several classification algorithms, including Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Neural Networks (NN). However, these models showed various limitations:

- 💡 SVM struggled significantly in handling the extreme class imbalance in the dataset. Its decision boundary became skewed toward the majority class, resulting in very poor recall for the minority (default) class, making it impractical in a high-stakes credit risk scenario for this highly imbalance dataset also the computational time very slow, around 40 minutes.
- 💡 KNN exhibited severe overfitting. While it performed reasonably on the training set, it failed to generalize on test data, especially underperforming in identifying defaults—most likely due to its sensitivity to local data structure and noise.
- 💡 Neural Network showed decent overall performance but lacked stability. The results fluctuated across runs due to sensitivity to initialization, and interpretability was limited, making it less actionable in financial contexts where model transparency is crucial.

📊**Model Performance Comparison: AUC & Fβ-score (β=3)**

Model	AUC	Fβ-score (β=3)
Logistic Regression	0.7817	0.1200
Random Forest	0.7985	0.1272

- We adopted a logistic regression model as the baseline. This model served as a solid reference point due to its simplicity, interpretability, and consistent performance. It achieved an Fβ-score (β=3) of 0.1200 and an AUC of 0.78 on the test set. As expected, it leaned toward conservative classification, leading to more false positives, which in credit risk is often more acceptable than false negatives.
- The Random Forest model was selected as the final model based on its robustness, ability to handle non-linear relationships, and built-in feature selection. Improved recall on default class while maintaining balanced performance across metrics. Despite the performance gain, the improvement over logistic regression is not dramatic. The increase in AUC is marginal (from 0.78 to 0.80), and the Fβ-score improved only slightly. This suggests that while Random Forest captures more complex patterns, the baseline model already performed competitively given the structured preprocessing and engineered features.
- Key considerations:

Random Forest Pros	Logistic Regression (Baseline) Pros
Better at capturing feature interactions.	**Highly interpretable**: Coefficients clearly show how features contribute to default.
Handles mixed-type data and non-linearities well.	**Transparent decision-making**: Useful in regulatory or financial reporting.
More robust against noise and outliers.	Easier to implement and maintain.

📌 **Given that the performance gap is small and logistic regression offers clearer insights into borrower behavior, the BASELINE MODEL remains a strong contender, especially when model explainability and actionable insights are prioritized.**

5. Discussion and Conclusion

Discussion: Interpretability and Insights

🔍 Final Takeaway Model

In this section, we provided a general overview of our final model, comparing its performance and interpretability with the baseline. While Random Forest offers marginally better performance, logistic regression provides superior interpretability, making it an excellent choice in high-stakes financial applications where transparency is key. Both models are reliable under the current setup, but the choice between them should weigh predictive power against the need for clear, explainable outcomes. *(See section Ultimate Model: Baseline vs Random Forest)*

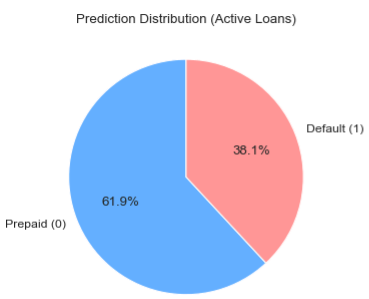
🔍 Predicting and Interpreting Predicted Risk on Active Loans

In this section, we explore the predictions made by our logistic regression model on the `d_active` dataset to better understand **where the predicted risk of default is concentrated**.

In [111…
```
#prediction from active data
na_counts = d_active.isna().sum()
na_columns = na_counts[na_counts > 0]
if not na_columns.empty:
    print("\n Columns with missing values:")
    print(na_columns)
else:
    print("\n No missing values in d_active.")

# Predizione diretta su d_active
d_active_pred = new_log_model.predict(d_active)

# Salva la predizione come nuova colonna
d_active['default_prediction'] = d_active_pred
d_active['default_probability'] = new_log_model.predict_proba(d_active)[:, 1]
```

```
No missing values in d_active.
```

In [112…
```
#computation for predicted default and prepaid
counts_active = d_active['default_prediction'].value_counts().sort_index()
labels = ['Prepaid (0)', 'Default (1)']
colors = ['#66b3ff', '#ff9999']
total_active = counts_active.sum()
percentages_active = (counts_active / total_active * 100).round(2)

summary_df_active = pd.DataFrame({'Prediction': labels,'Count': counts_active.values,'Percentage': percentages_active.values})

#test data (from logistics)
prepaid_count_test = total_count_test - default_count_test

percentage_default_test = 100 * default_count_test / total_count_test
percentage_prepaid_test = 100 - percentage_default_test

summary_df_test = pd.DataFrame({
    'Prediction': ['Prepaid (0)', 'Default (1)'],
    'Count': [prepaid_count_test, default_count_test],
    'Percentage': [round(percentage_prepaid_test, 2), round(percentage_default_test, 2)]
})

# === Plot: Pie + 2 tables ===
fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))

# Pie plot (active)
axes[0].pie(counts_active, labels=labels, autopct='%1.1f%%', startangle=90, colors=colors)
axes[0].set_title("Prediction Distribution (Active Loans)")

# Table 1: Active data
axes[1].axis('off')
table1 = axes[1].table(cellText=summary_df_active.values,
                       colLabels=summary_df_active.columns,
                       cellLoc='center',
                       loc='center')
table1.scale(1.2, 1.6)
axes[1].set_title("Summary: d_active")

# Table 2: Test set
axes[2].axis('off')
table2 = axes[2].table(cellText=summary_df_test.values,
                       colLabels=summary_df_test.columns,
                       cellLoc='center',
                       loc='center')
table2.scale(1.2, 1.6)
axes[2].set_title("Summary: Test Set")

plt.tight_layout()
plt.show()
```



Summary: d_active

Prediction	Count	Percentage
Prepaid (0)	45387	61.92
Default (1)	27908	38.08

Summary: Test Set

Prediction	Count	Percentage
Prepaid (0)	18094	71.42
Default (1)	7242	28.58

The pie chart shows the predicted distribution of outcomes for the **active loans** (`d_active`). According to the model's predictions:

- **38.1%** of the loans are expected to **default**
- **61.9%** are expected to be **prepaid**

This is a really big proportion of predicted defaults and may indicate that the model is identifying a significant portion of the active loans as risky. Compared to the performance of our model on the test set we see an incresing of 10% of predicted default, meaning that our model is detecting in the active dataset a significant higher number of risky loans.

It is important to compare this result with the **true historical distribution** of loan outcomes in the **training data**, where only **594 out of 101,343** loans actually defaulted, that is, just **0.6%**.

This contrast highlights that the model is **greatly overestimating the number of defaults** compared to the historical data.

Such behavior may be due to:

- The use of **recall-optimized metrics** (e.g., Fβ-score with β=3), which prioritize identifying defaults at the cost of precision
- The use of **oversampling techniques** (e.g., `RandomOverSampler`) during training to balance the rare default class

We may conclude that this tool is not primarily designed to **precisely predict** which individual loans will default and which will not. However, it still proves valuable for identifying **risk patterns** across the loans. The model can therefore be used effectively as a **screening tool**, flagging loans with riskier profiles that may warrant further investigation, either through human review or different type of analysis.

🔍 The Risk Signals: Predictive Insights from Active Loans

To gain perspective on the predictions from the active dataset, we draw comparisons with historical patterns observed in the original data. While we are **not comparing model predictions across the same dataset**, the striking difference between the **predicted default rate in** `d_active` **(~38%)** and the **observed default rate in the training set (~0.6%)** motivates us to use the initial dataset as a **reference**.

This comparison allows us to:

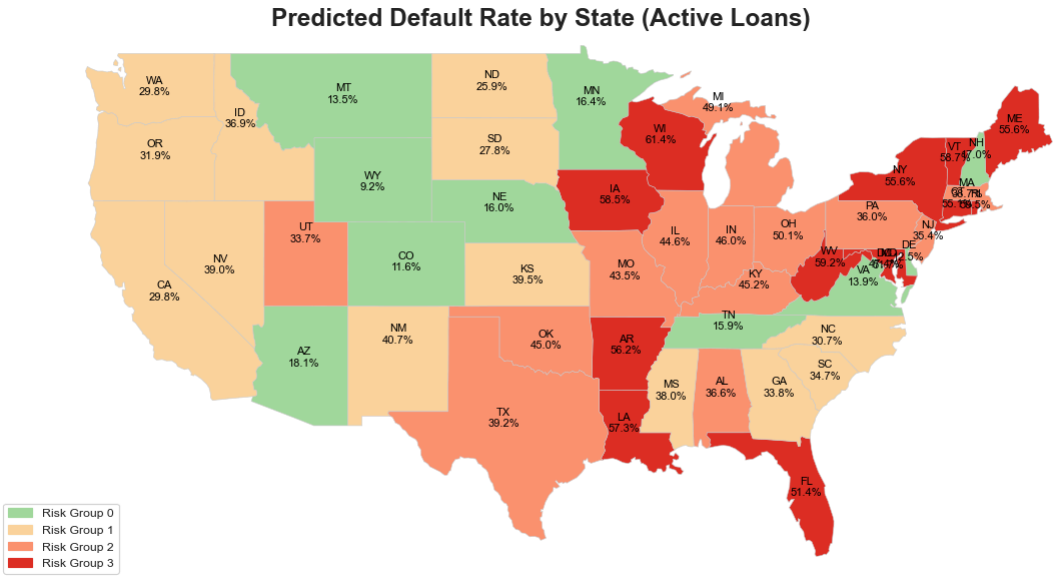
- Identify whether the model's predictions align with known risk profiles
- Detect possible biases or overestimations
- Evaluate how the model responds to specific features or flag combinations

Overall, the aim is to interpret the predictions **not as absolute truths**, but rather as **risk signals** that can guide further review or decision-making.

🗺️ A. Predicted Default Rate by State (Active Loans)

In [113…

```
# --- Compute predicted default rate by state using d_active ---
default_rate_df = ( d_active.groupby('st') .agg(total_loans=('default_prediction', 'count'),defaults=('default_prediction', 'sum'),
         st_group=('st_group', 'first')) .reset_index())

# Compute default rate percentage
default_rate_df['default_rate'] = 100 * default_rate_df['defaults'] / default_rate_df['total_loans']
default_rate_df = default_rate_df.rename(columns={'st': 'STUSPS'})

# --- Merge with GeoJSON ---
merged = us_states.merge(default_rate_df, on='STUSPS', how='left')

# --- Plot ---
fig, ax = plt.subplots(figsize=(16, 7.5))

# Define colors for each risk group
group_colors = {
    0: '#a1d99b',  # low risk
    1: '#fdd49e',  # low-medium risk
    2: '#fc9272',  # medium-high risk
    3: '#de2d26'   # high risk
}
merged['color'] = merged['st_group'].map(group_colors)

#map with custom colors
merged.plot(color=merged['color'],edgecolor='0.8',
    linewidth=0.5,ax=ax,missing_kwds={"color": "lightgrey", "label": "No data"})

#default rate labels to each state
for idx, row in merged.iterrows():
    if not pd.isna(row['default_rate']):
        point = row['geometry'].representative_point()
        label = f"{row['STUSPS']}\n{row['default_rate']:.1f}%"
        ax.text(point.x, point.y, label, ha='center', fontsize=10, color='black')  # Larger font

#continental US
ax.set_xlim(-130, -65)
ax.set_ylim(24, 50)

#title and legend
ax.set_title("Predicted Default Rate by State (Active Loans)", fontsize=22, fontweight='bold')
ax.axis('off')

#Legend for risk groups
import matplotlib.patches as mpatches
legend_patches = [mpatches.Patch(color=color, label=f"Risk Group {grp}") for grp, color in group_colors.items()]
plt.legend(handles=legend_patches, loc='lower left')
plt.tight_layout()
plt.show()
```



The map above visualizes the **predicted default rate** for active loans across U.S. states. Each state is colored according to its assigned **risk group** :

- **Risk Group 0**: lowest default rates (green)
- **Risk Group 1**: low-to-moderate default rates (orange)
- **Risk Group 2**: moderate-to-high default rates (light-red)
- **Risk Group 3**: highest default rates (red)

Key Observations:

- States in the **Midwest and Northeast**, such as **Iowa (58.5%)**, **Wisconsin (61.4%)**, **New York (55.6%)**, and **Vermont (58.7%)**, fall into the **highest risk group (Group 3)** with predicted default rates well above 50%.
- In contrast, several states in the **West and Midwest** such as **Wyoming (9.2%)**, **Colorado (11.6%)**, **Nebraska (16.0%)**, and **Virginia (13.9%)** exhibit **very low default rates** and are placed in **Risk Group 0**.
- Southern states like **Louisiana (57.3%)**, **Arkansas (56.2%)**, and **Florida (51.4%)** also show high predicted default rates
- Interestingly, large states like **California (29.8%)** and **Texas (39.2%)** fall into mid-range risk groups, indicating **mixed borrower profiles** or more stable predicted behavior.

This spatial analysis helps identify **geographic patterns of risk**, which can inform **regional lending policies**, targeted **risk monitoring**, or resource allocation for **collections and mitigation**. It also demonstrates that the model is sensitive to **location-based patterns**, potentially capturing economic, policy, or servicer-level effects.

Model vs Reality: Geographic Default Rate Comparison

When comparing the predicted default rates by state (last map) with the default rates observed in the training data (EDA), a clear discrepancy emerges.

In the **predicted map**, many states show extremely high default rates—often exceeding **50%**, in contrast, the distribution from the training data shows that even the riskiest states, such as **Louisiana** and **West Virginia**, had **actual default rates below 1.5%**, with the vast majority of states sitting well below **1%**.

A major limitation of our model is that it appears to **overestimate the influence of the borrower's state** on the likelihood of default.

This could be due to:

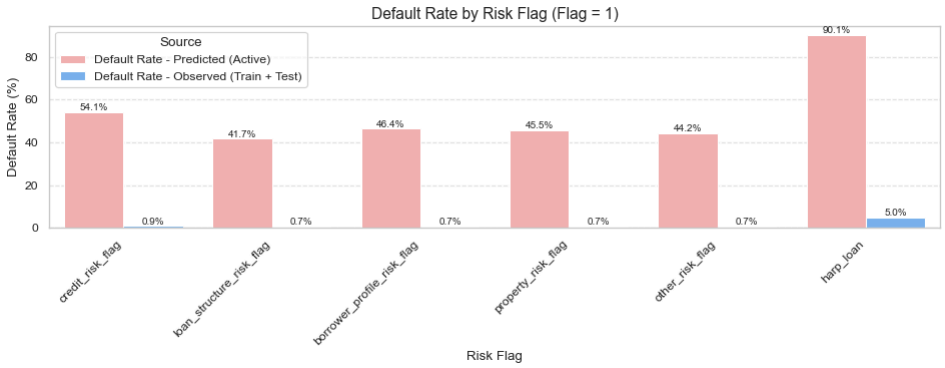
- The use of **engineered features like** `st_group`, which discretize risk by state
- **Overfitting** to regional patterns in the oversampled training set

This observation suggests that additional work is needed to **debias the model** and reassess how geographic information is used.

🔹 B. Default Rate by Risk Flag Value (Active vs. Train+Test)

In [114…
```
# Convert 'harp_loan' from 'Y'/'N' to 1/0 in both active and new binary dataset (original dataset train + test)
d_active['harp_loan'] = d_active['harp_loan'].map({'Y': 1, 'N': 0})
d_binary['harp_loan'] = d_binary['harp_loan'].map({'Y': 1, 'N': 0})
```

In [115…
```
# flags used in the logistic
flags = [ 'credit_risk_flag', 'loan_structure_risk_flag', 'borrower_profile_risk_flag', 'property_risk_flag', 'other_risk_flag','harp_loan']

comparison_data = []

for flag in flags:
    # Predicted defaults in active set
    active_subset = d_active[d_active[flag] == 1]

    default_rate_active = 100 * active_subset['default_prediction'].mean()

    # Observed defaults in train set
    train_subset = d_binary[d_binary[flag] == 1]
    default_rate_train = 100 * train_subset['loan_status'].mean()

    #add to list
    comparison_data.append({'Flag': flag,
                            'Default Rate - Predicted (Active)': round(default_rate_active, 2),
                            'Default Rate - Observed (Train + Test)': round(default_rate_train, 2) })

# dataframe for plot
comparison_df = pd.DataFrame(comparison_data)
df_long = comparison_df.melt(id_vars='Flag', var_name='Source', value_name='Default Rate (%)')

#col and labels
palette = {'Default Rate - Predicted (Active)': '#fca4a4','Default Rate - Observed (Train + Test)': '#66b3ff'}

# plot
plt.figure(figsize=(12, 4.8))
ax = sns.barplot(data=df_long, x='Flag', y='Default Rate (%)', hue='Source', palette=palette)

for p in ax.patches:
    height = p.get_height()
    if height > 0.1: #to avoid some weird percentage coming up
        ax.annotate(f'{height:.1f}%',
                    (p.get_x() + p.get_width() / 2., height),
                    ha='center', va='bottom', fontsize=9)

plt.title("Default Rate by Risk Flag (Flag = 1)", fontsize=14)
plt.ylabel("Default Rate (%)")
plt.xlabel("Risk Flag")
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



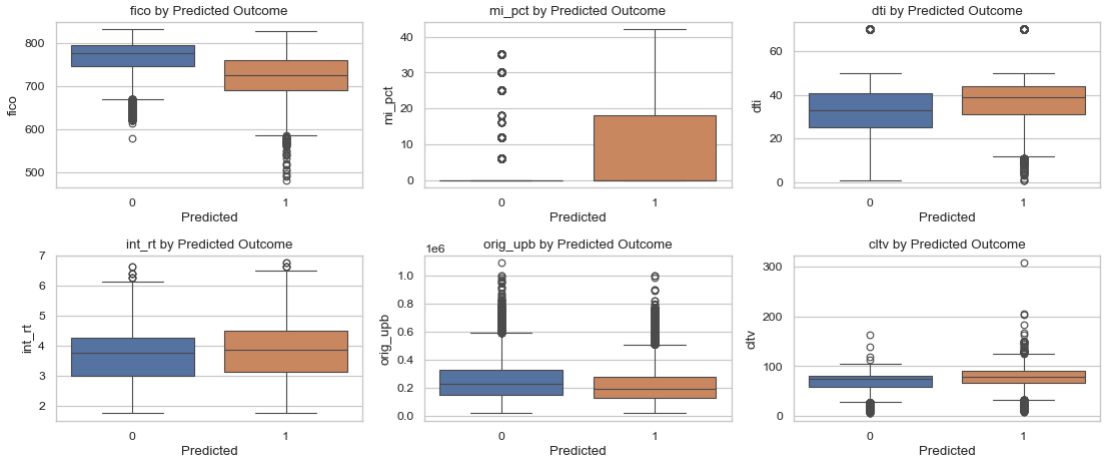
The bar plots above compare the **default rate** observed in the training+test dataset and the **predicted default rate** on the `d_active` dataset, when a given binary **risk flag** is active (`flag = 1`).

- For all flags, the **model severely overestimates the default rate** in the active loans.
  - *Example*: For `harp_loan`, the predicted default rate is **90.1%**, while in the training+test data it is only **5.0%**.
- Despite this overestimation, the **relative ranking of risk severity** is largely preserved:
  - Flags associated with higher observed risk (like `harp_loan` and `credit_risk_flag`) also show higher predicted rates.

These results emphasize that while the model captures **relative risk levels**, it is **not calibrated in absolute terms**. The percentage of predicted defaults on the active are significantly higher than the observed data, which is consistent with earlier findings.

🔹 C. Distribution of Numerical Features by Predicted Outcome

In [116…
```
# Variabili numeriche
numerical_cols = ['fico', 'mi_pct', 'dti', 'int_rt', 'orig_upb', 'cltv']

# Imposta lo stile
sns.set(style="whitegrid")

# Numero di colonne per subplot
n_cols = 3
n_rows = -(-len(numerical_cols) // n_cols)  # ceil division

# Crea sottografi
fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 3 * n_rows))
axes = axes.flatten()

for i, col in enumerate(numerical_cols):
    sns.boxplot(
        data=d_active,
        x='default_prediction',
        y=col,
        hue='default_prediction',
        legend = False,
        ax=axes[i]
    )
    axes[i].set_title(f'{col} by Predicted Outcome')
    axes[i].set_xlabel('Predicted')
    axes[i].set_ylabel(col)

# Rimuove grafici vuoti (se presenti)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



The boxplots above visualize the distribution of key numerical variables in the `d_active` dataset, comparing loans predicted to **default** (`1`) vs. those predicted as **prepaid** (`0`).

Observations:

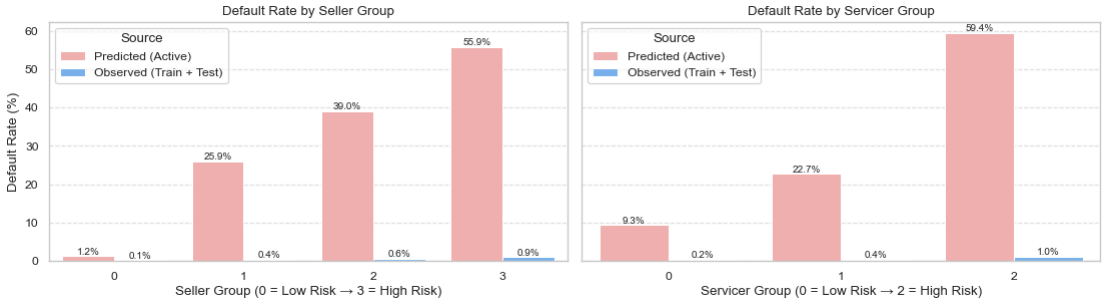
- **fico**: Loans predicted as defaulted have significantly **lower credit scores** compared to prepaid ones. This is consistent with domain expectations: lower FICO → higher credit risk.
- **mi_pct**: Mortgage insurance is **almost always 0** for prepaid loans, while defaulted predictions show **a much higher spread**, with many values well above 10–20%. This supports the idea that loans requiring insurance are viewed as riskier.
- **dti (Debt-to-Income ratio)**: Median is **higher for predicted defaults**, suggesting that higher DTI is associated with increased risk.
- **int_rt (Interest Rate)**: Slight shift upward in the interest rate for defaults, though the difference is more subtle.
- **orig_upb (Original Loan Amount)**: Surprisingly, there is **no major difference** in median loan amount
- **cltv (Combined Loan-to-Value ratio)**: As expected, loans predicted to default tend to have **higher CLTV**, meaning borrowers owe more relative to the value of their home — a key credit risk indicator.

These plots confirm that the model is picking up on **real risk signals** embedded in the numerical variables. In particular, **fico**, **cltv**, and **dti** show strong and separations between predicted outcomes.

📘 D. Default Rate by Seller and Servicer Group

In [117…

```
# default rate per group
def compute_group_default_rate(df, group_col, target_col):
    stats = (df.groupby(group_col)[target_col].mean().reset_index()
        .rename(columns={target_col: 'Default Rate (%)'}))
    stats['Default Rate (%)'] *= 100
    stats[group_col] = stats[group_col].astype(str)
    return stats

#seller group
seller_active = compute_group_default_rate(d_active, 'seller_name_group', 'default_prediction')
seller_train = compute_group_default_rate(d_binary, 'seller_name_group', 'loan_status')
seller_active['Source'] = 'Predicted (Active)'
seller_train['Source'] = 'Observed (Train + Test)'
seller_combined = pd.concat([seller_active, seller_train])

#servicer group
servicer_active = compute_group_default_rate(d_active, 'servicer_name_group', 'default_prediction')
servicer_train = compute_group_default_rate(d_binary, 'servicer_name_group', 'loan_status')
servicer_active['Source'] = 'Predicted (Active)'
servicer_train['Source'] = 'Observed (Train + Test)'
servicer_combined = pd.concat([servicer_active, servicer_train])

# Plotting
fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)

#seller
ax1 = sns.barplot(data=seller_combined, x='seller_name_group', y='Default Rate (%)',
                  hue='Source', palette=['#fca4a4', '#66b3ff'], ax=axes[0])
axes[0].set_title("Default Rate by Seller Group")
axes[0].set_xlabel("Seller Group (0 = Low Risk → 3 = High Risk)")
axes[0].set_ylabel("Default Rate (%)")
axes[0].grid(axis='y', linestyle='--', alpha=0.5)

# % eller
for p in ax1.patches:
    height = p.get_height()
    if height > 0:
        ax1.annotate(f'{height:.1f}%',
                     (p.get_x() + p.get_width() / 2., height),
                     ha='center', va='bottom', fontsize=9)

#servicer
ax2 = sns.barplot(data=servicer_combined, x='servicer_name_group', y='Default Rate (%)',
                  hue='Source', palette=['#fca4a4', '#66b3ff'], ax=axes[1])
axes[1].set_title("Default Rate by Servicer Group")
axes[1].set_xlabel("Servicer Group (0 = Low Risk → 2 = High Risk)")
axes[1].grid(axis='y', linestyle='--', alpha=0.5)

# % servicer
for p in ax2.patches:
    height = p.get_height()
    if height > 0:
        ax2.annotate(f'{height:.1f}%',
                     (p.get_x() + p.get_width() / 2., height),
                     ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```



The plots above show the **default rate by grouped seller and servicer identifiers**, both in:

- **Observed data** (Train + Test, in blue)
- **Predicted outcomes** for `d_active` (in red)

Each group (0 to 2 or 3, depending on the case) reflects an ordinal transformation of the original categorical variables, based on their observed default rate in the training data:

- **Group 0 = Lowest risk**
- **Group 3 = Highest risk**

Key Insights:

- The model clearly **preserves the intended risk ordering**, with default rates increasing across the groups for both sellers and servicers.
- However, we again observe a **strong overestimation** of default risk in the active dataset:
  - For example, in **Seller Group 3**, the predicted default rate is **55.9%**, compared to just **0.9%** observed in the historical data.
  - Similarly, **Servicer Group 2** shows a predicted rate of **59.4%**, while the actual observed rate is **1.0%**.

This suggests that while the model **captures relative differences in risk across categorical groupings**, it tends to **increse the absolute probability of default**.

✅ **General Insight: Analysis of Active Loan Predictions**

The analysis of the `d_active` dataset revealed a clear pattern: our model tends to **exaggerate risk signals** present in the data. While it successfully captures the **direction and the relative risk** of various risk flags, state-level risk groups, and seller/servicer categories, it **significantly increases the predicted default rates** , compared to historical data.

For example, features associated with higher observed default rates in the training data (e.g., low FICO scores, cash-out loans, or high-risk servicer groups) also show higher predicted default rates in `d_active` , preserving their **relative importance**. However, the **absolute levels** of predicted default are often much higher than those observed historically.

This over-amplification limits the model's reliability in producing **calibrated, loan-level predictions**, making it less suitable for exact default probability estimation or binary classification decisions. Instead, the model is better interpreted as a **risk screening tool**: it can help identify loans or segments of the portfolio that **deserve closer attention or further review**, either by human analysts or more specialized decision systems.

High-Risk Sellers, Servicers, and States

In order to inform Freddie Mac of potential risk categories, we display the highest-risk categories for each group: sellers, servicers, and states.

In [118…
```
# don't delete, useful later at the end (high risk groups)
sellers_group_3 = X_train.loc[X_train['seller_name_group'] == 3, 'seller_name'].unique()
states_group_3 = X_train.loc[X_train['st_group'] == 3, 'st'].unique()
servicers_group_2 = X_train.loc[X_train['servicer_name_group'] == 2, 'servicer_name'].unique()
zip_group_1 =  X_train.loc[X_train['zipcode_group'] == 1, 'zipcode'].unique()
```

In [119…
```
#display risky categories (computed before)
sellers_series = pd.Series(sellers_group_3, name='High-Risk Sellers (Group 3)')
states_series = pd.Series(states_group_3, name='High-Risk States (Group 3)')
servicers_series = pd.Series(servicers_group_2, name='High-Risk Servicers (Group 2)')

# Concatenate into a single DataFrame (aligning by index)
risk_table = pd.concat([sellers_series, states_series, servicers_series], axis=1)
risk_table_clean = risk_table.fillna("")


display(risk_table_clean)

#zip_group_1
```

High-Risk Sellers (Group 3)	High-Risk States (Group 3)	High-Risk Servicers (Group 2)

Zipcode risky catefories are saved in `zip_group_1` and not displayed beacuse there are high number of them.

Business Recommendation

The following business recommendations are particularly noteworthy and warrant further consideration:

- **1. Intensify Oversight on High-Risk Sellers and Servicers**
  The model consistently flags certain seller and servicer groups—particularly Group 3 sellers and Group 2 servicers—as significantly more prone to defaults. These entities likely engage in higher-risk origination practices, such as underwriting loans with limited documentation or catering to subprime markets. Regulatory oversight and policy adjustments should focus on these institutions, such as enforcing stricter capital buffers, regular performance audits, or limiting loan volumes temporarily. Elul et al. (2010) emphasize that institutional behavior is a major driver of mortgage performance, beyond borrower attributes.

- **2. Implement Risk-Adjusted Lending Standards by Region**
  High-risk regions (e.g., State Group 3 and ZIP Group 1) may face economic instability, weaker job markets, or slower housing recovery rates. Mayer & Pence (2008) show that local economic factors and legal environments, like foreclosure laws, significantly affect default trends. Freddie Mac should apply more stringent approval criteria, enforce full property appraisal requirements, or adjust LTV thresholds in these areas to better manage geographic concentration risk.

- **3. Apply Conservative Lending to High-Risk Borrower Profiles**
  Borrowers identified through engineered risk flags—such as first-time homebuyers, single borrowers, and those with high DTI—have been shown to be more vulnerable to financial stress. Studies like Deng et al. (2000) highlight that such borrowers typically lack the buffers needed to absorb economic shocks. Freddie Mac should promote educational programs, offer step-up repayment schedules, or require enhanced credit enhancements (e.g., MI or reserves) for these segments.

- **4. Use Model Scores to Trigger Early Warning Systems on Active Loans**
  Rather than only using credit scores at origination, the model can be used as an early warning tool to continuously score active loans. Loans that start resembling the high-risk patterns of past defaulters can be flagged, allowing the company to intervene early—such as by offering refinancing options, restructuring terms, or initiating soft outreach. This aligns with preventive risk management best practices and supports long-term portfolio stability.

- **5. Encourage Targeted Refinancing for High-Risk Loans**
  While HARP and other modification programs aim to reduce risk, some loans still exhibit elevated default rates post-modification. Agarwal et al. (2012) suggest that refinancing programs are most effective when coupled with borrower screening and post-refinance monitoring. Freddie Mac can enhance program impact by targeting refinances toward borrowers flagged as high risk by the model—while avoiding unnecessary refinancing for already low-risk borrowers, preserving resources.

- **6. Integrate Interpretable Models for Regulatory and Client Communication**
  Logistic regression, while simpler, provides coefficients that clearly explain why a borrower is considered risky. This is invaluable for financial institutions required to disclose reasons for adverse decisions or to comply with fair lending practices. Bhattacharya et al. (2019) stress the importance of model interpretability in maintaining trust with regulators and borrowers. A dual-model strategy—complex model for prediction, simple model for explanation—can maximize both performance and transparency.

- **7. Redesign Product Offerings Based on Risk Insights**
  The model highlights structural loan characteristics (e.g., high CLTV, short terms, cash-out refinancing) as major contributors to risk. These insights can guide Freddie Mac's product design. For instance, limiting cash-out refinance LTVs, incentivizing longer amortization schedules, or promoting MI-backed loans could reduce defaults. Bhardwaj & Sengupta (2001) show that product type and structure significantly affect borrower incentives and repayment behavior.

Limitations and Future Directions

Before relying on any predictive model for high-stakes decision-making such as mortgage default assessment, it is essential to reflect on both its current limitations and future directions for improvement. While the model developed in this project demonstrates encouraging results in identifying potential default risks, no model is without trade-offs. A clear understanding of its constraints will help guide responsible usage and continuous refinement over time.

The following section outlines key limitations that may affect the model's robustness, interpretability, or generalizability, followed by practical suggestions for future enhancements grounded in research and industry practices.

Limitation	Future Work
Static Economic Context The model is built on historical loan data and does not account for future changes in macroeconomic conditions such as inflation, housing downturns, or interest rate shocks. Since mortgage performance is highly sensitive to such variables, this limits the model's ability to generalize across different economic cycles.	**Incorporate Macroeconomic Indicators** To better anticipate defaults, future models could integrate external macroeconomic data like regional unemployment, housing price indices, or mortgage rates. These signals have been shown to significantly affect borrower behavior (Mayer & Pence, 2008).
Severe Class Imbalance Despite applying random oversampling to rebalance the dataset, the original 0.6% default rate introduces substantial challenges. Models trained on imbalanced data tend to favor the majority class, potentially underperforming when applied to rare default events.	**Use Advanced Resampling Techniques** Synthetic methods such as SMOTE or ensemble balancing can generate more meaningful minority class examples, potentially improving model robustness without duplicating existing data (Chawla et al., 2002).
Limited Granularity in Risk Flags While aggregated flags simplify modeling, they can obscure important nuances in individual borrower risk. This tradeoff between interpretability and precision may limit the model's ability to differentiate between borrowers who fall just above or below a risk threshold.	**Apply Explainable AI (XAI) Tools** Incorporating tools like SHAP or LIME would provide localized explanations for each loan prediction, aiding credit officers and building trust with regulatory bodies (Lundberg & Lee, 2017).
Dependence on Training-Period Groupings Categorical features like `seller_name` and `st` were grouped by historical default rate. However, these groupings may shift over time due to policy changes, economic shifts, or market dynamics—making the model less robust in evolving environments.	**Reevaluate Risk Group Assignments Periodically** Since categorical groupings (e.g., `seller_name_group`) are based on default history, periodic retraining is needed to ensure group definitions remain valid over time (Davis & Goadrich, 2006).
Low Precision in Default Class The model improves recall for default predictions, but at the cost of precision. This means many flagged defaults may be false alarms, increasing the risk of overreaction or inefficient loan servicing strategies.	**Integrate Behavioral Features** Including variables like delinquency trends or early missed payments may allow for earlier detection of deteriorating loan health. Behavioral predictors have proven valuable in enhancing early-warning systems (Agarwal et al., 2012).

Conclusion

This project set out to build a model that could help identify which mortgage loans are most at risk of default. By carefully analyzing historical loan data from Freddie Mac, we discovered that certain borrower characteristics—such as low credit scores, high debt levels, and specific loan types—are closely linked with a higher chance of default. We started with a simpler model that provided clear and understandable results, and then tested a more advanced model to see if performance could be improved. While the advanced model showed slightly better results, the simpler model still performed well and was easier to interpret, making it valuable for practical decision-making.

From this work, several important insights emerged that can help organizations better manage loan risk. Certain institutions and regions were repeatedly linked to higher defaults, and borrowers with specific risk profiles stood out as needing closer attention. By applying the model to current loans, we were also able to flag which ones might be at higher risk moving forward—enabling earlier action and better resource allocation. Overall, this project highlights how data-driven tools can support more informed, fair, and proactive decisions in the mortgage lending process.

6. Generative AI statement

Generative AI tools were used responsibly throughout this project to support and enhance the analysis process. Specifically, they were used in the following ways:

- Used for debugging code errors, assisting in identifying and fixing technical issues during model development.
- Helped clarify unfamiliar terms or concepts related to the mortgage and loan industry.
- Provided feedback on analysis, including interpretation and suggestions for improvement.
- Utilized to check grammar and spelling throughout the final report to ensure clarity and professionalism.

7. References

- Agarwal, S., Amromin, G., Ben-David, I., Chomsisengphet, S., & Evanoff, D. D. (2012). Policy intervention in debt renegotiation: Evidence from the Home Affordable Modification Program. *Journal of Political Economy*, 120(3), 653–691. Link
- Avery, R. B., Bostic, R. W., Calem, P. S., & Canner, G. B. (2004). Credit risk, credit scoring, and the performance of home mortgages. *Federal Reserve Bulletin*, 90(9), 621–648. Link
- Avery, R. B., Bhutta, N., Brevoort, K. P., & Canner, G. B. (2019). The 2015 HMDA data: Mortgage market activity and the affordability gap. *Federal Reserve Bulletin*, 105(3). Link
- Bhardwaj, G., & Sengupta, R. (2020). Where's the risk? The mortgage credit risk transfer market and the effects of credit characteristics. *Federal Reserve Bank of St. Louis Review*, 102(3), 223–244. Link
- Bhutta, N., Fuster, A., & Hizmo, A. (2020). Paying too much? Price dispersion in the US mortgage market. *Finance and Economics Discussion Series*, 2020-062. Link
- Bhutta N, Aurel Hizmo.(2021). Do Minorities Pay More for Mortgages?, The Review of Financial Studies, Volume 34, Issue 2, February 2021, Pages 763–789, Link
- Campbell, J. Y., Jackson, H. E., Schoar, A., & Tufano, P. (2011). Consumer Financial Protection. Journal of Economic Perspectives, 25(1), 91–114. Link
- CFPB. (2022). Ability-to-Repay and Qualified Mortgage Standards under the Truth in Lending Act. Link
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. Link
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). Link
- Deng, Y., Quigley, J. M., & Van Order, R. (2000). Mortgage terminations, heterogeneity and the exercise of mortgage options. *Econometrica*, 68(2), 275–307. Link
- Elul, R., Souleles, N. S., Chomsisengphet, S., Glennon, D., & Hunt, R. (2010). What "triggers" mortgage default? *American Economic Review*, 100(2), 490–494. Link
- Foote, C. L., Gerardi, K., & Willen, P. S. (2008). Negative equity and foreclosure: Theory and evidence. *Journal of Urban Economics*, 64(2), 234–245. Link
- Frame, W. S., Lin, E. Y., & White, L. J. (2007). The roles of GSEs in the mortgage market. *Federal Reserve Bank of Atlanta Economic Review*, 92(1), 87–103. Link
- Gerardi, K., Shapiro, A. H., & Willen, P. S. (2013). Subprime outcomes: Risky mortgages, homeownership experiences, and foreclosures. *NBER Working Paper No. 18407*. Link
- He, H., & Garcia, E. A. (2009). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. Link
- Jagtiani, J., & Lemieux, C. (2017). The roles of alternative data and machine learning in fintech lending: Evidence from the LendingClub consumer platform. *Federal Reserve Bank of Philadelphia Working Paper*. Link
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (pp. 4765–4774). Link
- Mayer, C., & Pence, K. (2008). Subprime mortgages: What, where, and to whom? *Housing Policy Debate*, 19(3), 457–486. Link
- Quercia, Roberto G. and Pennington-Cross, Anthony N. and Tian, Chao Yue. (2012). Mortgage Default and Prepayment Risks Among Moderate and Low Income Households (July 25, 2011). Real Estate Economics, Forthcoming, Link
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. Link
- Van Dyk, D. A., Park, T., & Meng, X.-L. (2008). Partially collapsed Gibbs samplers: Illustrations and applications. *Journal of Computational and Graphical Statistics*, 17(1), 1–22. Link

In [124…
```
# Run the following to render to PDF
!jupyter nbconvert --to html project_mortgage_ml.ipynb
```

```
[NbConvertApp] Converting notebook project_mortgage_ml.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 32 image(s).
[NbConvertApp] Writing 3017280 bytes to project_mortgage_ml.html
```